



M362 Unit 5

UNDERGRADUATE COMPUTING

Developing concurrent distributed systems



Distributed systems:
Software architecture
and communication

Unit **5**

This publication forms part of an Open University course M362 *Developing concurrent distributed systems*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 1593 4

1.1

1	Introduction	6
1.1	The aims of this unit	6
2	Software architectures for distributed systems	7
2.1	Client–server style	7
2.2	Multiple servers acting together	8
2.3	Proxy servers	10
2.4	Peer-to-peer style	11
3	Tiered architectures for client–server systems	14
3.1	Traditional two-tier model	14
3.2	Layered architectures: interfaces and components	16
3.3	Musical instruments	19
3.4	The n-tier architecture	19
4	Java Enterprise Edition	23
4.1	What is an enterprise?	24
4.2	The Java Platform, Enterprise Edition	25
5	Distributed communications – message passing	28
5.1	Synchronous and asynchronous communication	28
5.2	Timeouts and deadlock	30
5.3	Addressing	31
5.4	Data in message passing	32
6	Client–server communication in Java	35
6.1	Sockets	35
6.2	The <code>java.net</code> package	36
6.3	A simple ‘date and time’ server	37
6.4	A chat room server	39
7	Paradigms for distributed communication	47
7.1	Remote procedure call (RPC)	47
7.2	Distributed object paradigm	51

Continued over page

8	Advanced forms of communication	55
8.1	Object request brokers	55
8.2	The messaging system paradigm	56
8.3	Publish/subscribe messaging model	56
8.4	Peer-to-peer file sharing	57
8.5	Trade-offs	59
9	Summary	61
	Glossary	63
	References	68
	Acknowledgements	68
	Index	69



M362 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Chair, author and academic editor

Janet van der Linden

Authors

Anton Dil

Brendan Quinn

Michel Wermelinger

Critical readers and testers

Henryk Krajinski

Barbara Segal

Mark Thomas

Richard Walker

Yijun Yu

External assessor

Aad van Moorsel, Newcastle University

Software development

Ivan Dunn, Consultant

Course management

Linda Landsberg

Carrie Lewis

Barbara Poniatowska

Julia White

Media development staff

Ian Blackham, Editor

Sarah Gamman, Contracts Executive

Jennifer Harding, Editor

Phillip Howe, Media Assistant

Martin Keeling, Media Assistant

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Sue Stavert, Technical Testing Team

Andrew Whitehead, Designer and Graphic Artist

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics, Computing and Technology.

1

Introduction

In this unit we introduce the topic for Block 2 of this course, which is distributed systems. In *Unit 1* we defined a distributed system as a system consisting of a number of component parts that are located in distinct places. These components are connected, which allows them to communicate and coordinate their actions. The software components of a distributed system can be clustered in different ways, described by so-called software architectures, and some are more able than others to cope with the following requirements for distributed systems.

- ▶ If part of the system fails, possibly due to a hardware fault, the system as a whole should be able to continue its work.
- ▶ If many users decide to use the system simultaneously, the system should be able to cope with this *and* be reasonably responsive.

We will begin with an outline of different ways of clustering the parts of a distributed system, and compare these in terms of the above requirements.

The various functions performed by distributed systems can be separated from each other. This will be discussed and illustrated with the case study of the Music Shop, as it develops from a shop with relatively straightforward computing requirements, to a more complicated type of business with more advanced computing requirements.

Another important requirement for distributed systems is the ability for one part of the system to communicate with another part. The most basic form of communication is message passing and Java has a number of classes that allow us to program this form of communication.

There are also a number of more advanced forms of communication. Given that software technologies are changing so rapidly in this area, we will concentrate on the core issues, although later units will add more details of some of these approaches.

1.1 The aims of this unit

In this unit you will study:

- ▶ software architectures for distributed systems, and the development from two-tier to n-tier systems (Sections 2 and 3);
- ▶ the concept of an enterprise, and the Java Enterprise Edition (Java EE) framework (Section 4);
- ▶ message passing as a basic form of communication, and how to write simple client–server systems in Java (Sections 5 and 6);
- ▶ higher-level forms of communication (Sections 7 and 8).

This unit is an introduction to Block 2 of the course as a whole, and you may find that some of the concepts go by a bit quickly. Don't worry, as these will be explained in more detail in later units.

This unit mainly involves reading together with exercises (including one exercise that involves internet research), apart from Section 4 and Section 6 (the latter containing some practical activities on the implementation of client–server systems in Java).

2

Software architectures for distributed systems

In this section we will discuss the main **software architectures** for distributed systems. A software architecture is concerned with the structure or structures of the system, which comprise software elements, the main properties of those elements, and the relationships between them

The software architectures use *processes* as the elements or components to build on. We will distinguish the main function and responsibility of different types of process, and then consider how placing these processes in a network can create useful architecture styles for the distribution of data and workload, and styles of communication between the processes.

We will discuss four different software architecture styles that are important for distributed systems: client–server, multiple servers, proxy server and peer-to-peer.

In this section we will concentrate on fault tolerance and performance issues for each style of software architecture. **Fault tolerance** is defined as the ability of a system to respond gracefully to an unexpected hardware or software failure. By 'gracefully' we mean that if a system's operating quality decreases at all due to a failure, the decrease is proportional to the severity of the failure. This is in contrast to a naively designed system in which even a small failure can cause total breakdown. The **performance** of a distributed system can be considered under the following headings.

- ▶ **Responsiveness.** Generally speaking, if one part of the system needs to wait a long time before it gets a response from another part, the performance of the system is considered inadequate. For users of interactive applications, poor responsiveness is particularly frustrating and can make the system unusable.
- ▶ **Throughput.** The performance of a system can also be expressed in terms of its throughput, that is, the rate at which the work is done (for example, the number of transactions per second), and whether the system performs work for all its users or only a few. The throughput is influenced by the processing speeds of the various software components, the reliability of the network connections, and all the other delays involved at each layer of software through which the data will have to pass as it is transferred.

2.1 Client–server style

Perhaps the best-known distributed software architecture is the **client–server** style in which asymmetric roles are assigned to two collaborating processes. One process, the server, plays the role of a service provider which waits passively for the arrival of requests. The other, the client, issues specific requests to the server and awaits its responses. This is shown in Figure 1.

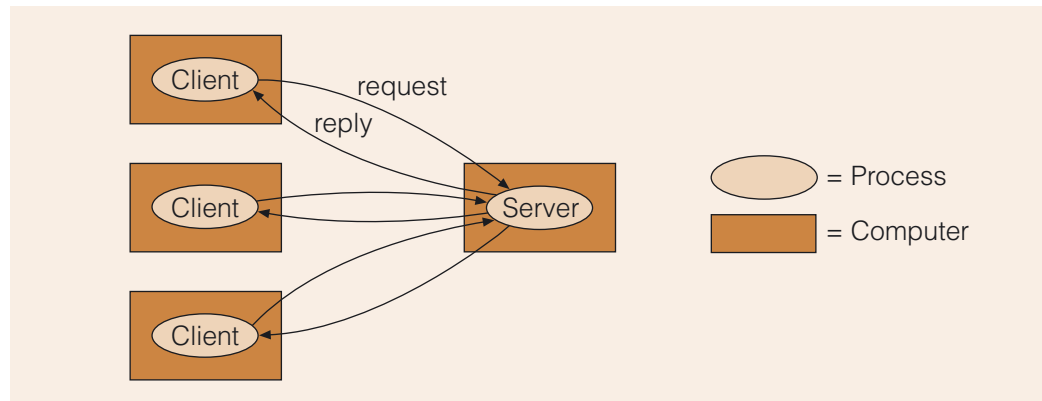


Figure 1 Client-server style

Figure 1 presents a view of the client-server software architecture, based on which component runs on which host. This is sometimes referred to as a physical view of the software architecture. It shows that the server process is normally placed on a different host from the client process and typically manages large amounts of data and files. Many clients can connect to one server.

In terms of performance, the simple client-server style has one obvious bottleneck: if many clients are connected to a single server, then the responsiveness of the system as a whole is likely to decrease.

To some extent this problem can be addressed by using the processing power of the clients. For example, it might be possible that clients are made to download and run an applet from the server, thus avoiding the need to wait for the server machine to do all the processing. This is an example of **load balancing** – avoiding situations where one component has to do all the work, while other components remain idle. Load balancing is an important aim of distributed systems, enabling the available computational resources to be better utilised.

However, even with the clients doing a significant amount of processing, the central role of the server remains the weakness of this software architecture, both from a performance viewpoint and due to its vulnerability in a case of failure. We say that the server is the system's **single point of failure**: if the server goes down, the whole system goes down.

2.2 Multiple servers acting together

There are a number of variations on the classic client-server style. One of these is where a server acts both in the role of server and in that of client, as shown in Figure 2.

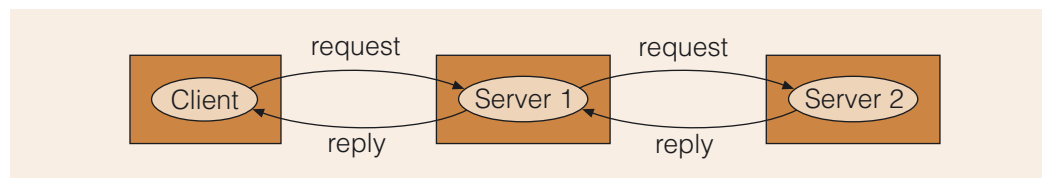


Figure 2 Client-server model: server as a client

In Figure 2 we see an example of a client process sending a request message to a server, labelled server 1. This server in turn sends a request message to a different server, server 2, which carries out part of the processing. The reply message that is returned from server 2 is processed by the first server, after which it sends a reply message to the client.

We will study applets in Unit 9.

This scenario could take place, for example, in the situation where a client contacts a server with a request for a price comparison for a product, say flights for a holiday destination. This server will need to send requests for information to the servers of different flight companies in order to collect the data. The server conducting the price comparison acts as a client towards the servers that supply it with information.

In the system architecture of Figure 2 the two servers are acting together and cooperating, but in a rather loose way. A more structured and organised way for multiple servers to act together is illustrated in Figure 3.

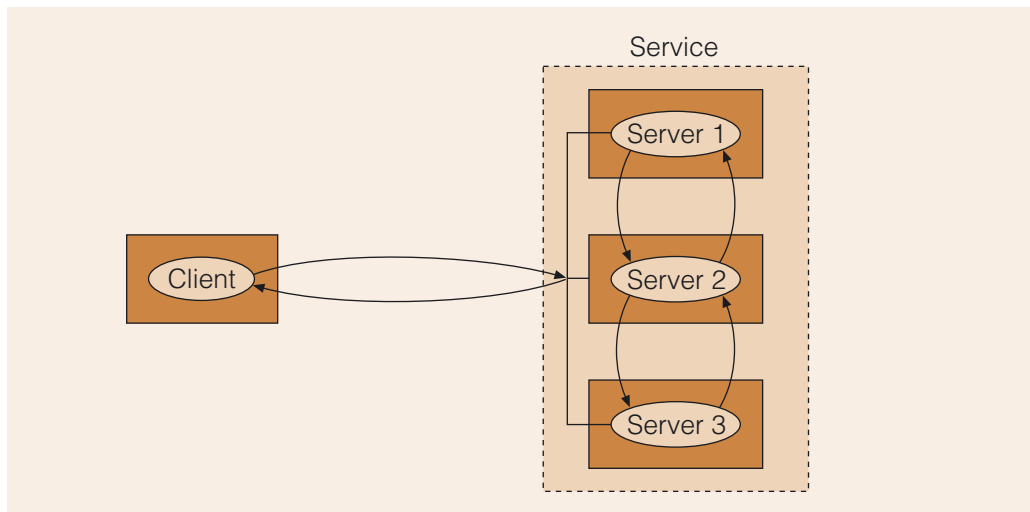


Figure 3 A service provided by a number of servers

Figure 3 shows a client contacting a server, where the server works closely together with a number of other servers to provide a service. There are two main ways in which the system architecture in Figure 3 may be implemented:

- ▶ *partitioning* the data among the servers;
- ▶ *replicating* the data among the servers.

With **data partitioning**, the data resources for the system as a whole are distributed among the servers, such that each server is responsible for its own section or partition. The servers interact with each other to provide the replies to requests.

Data partitioning would be used by large international commercial organisations, such as Amazon or eBay, offering online services. For example, the account details for customers could be held on the server local to each country, as well as the availability of items for that country. However, when clients issue request messages for information about items, they will receive a response detailing both the items that are available within the country and those available from other countries.

With **data replication**, each server holds an identical copy of the data. This may be done for reasons of better performance and availability, as well as improved fault tolerance.

We illustrate the idea of data replication through the example of an online banking system where, instead of having one central database, the database has been replicated to run on a number of separate servers.

- ▶ The performance of such an online banking system can be improved because geographically dispersed clients can now contact a server that is more local to them, hence reducing the communication delay.

- ▶ Should one of the servers fail, all the clients that accessed this server can be diverted to another server, thus providing better availability. This might take some extra time, but it does mean that a more reliable service can be established.
- ▶ The malfunctioning of a server can be detected if all the operations on the data are mirrored for each server and the results are compared. For example, when a query on a customer's account is made, this query can be made to all the servers concurrently. The results from every query can be compared: if a server is malfunctioning intermittently, one of the answers to the query might be incorrect. Monitoring software can pick this up and can carry out appropriate actions.

Note that the above example demonstrates two types of fault tolerance. First, it considers the possibility of one of the server machines going down completely, and its job being taken over by another server machine, such that the user need never be aware of the change. Secondly, we considered whether it would be possible to detect the malfunctioning of a server which may not have failed completely. The second type of malfunctioning is more difficult to spot, and may have gone on for some time before it is detected. The added complexity in coping with this type of fault means that the necessary fault tolerance may only be built into systems where the nature of the application requires that such scenarios should not be allowed to go undetected.

2.3 Proxy servers

The word proxy is generally used for a person acting on behalf of another person – for example, in elections a voter can choose to have their vote cast by proxy, which means that they would normally appoint someone they trust to act as their proxy. In computing, a **proxy server** is a server that acts as an intermediary between clients and other servers, as illustrated in Figure 4.

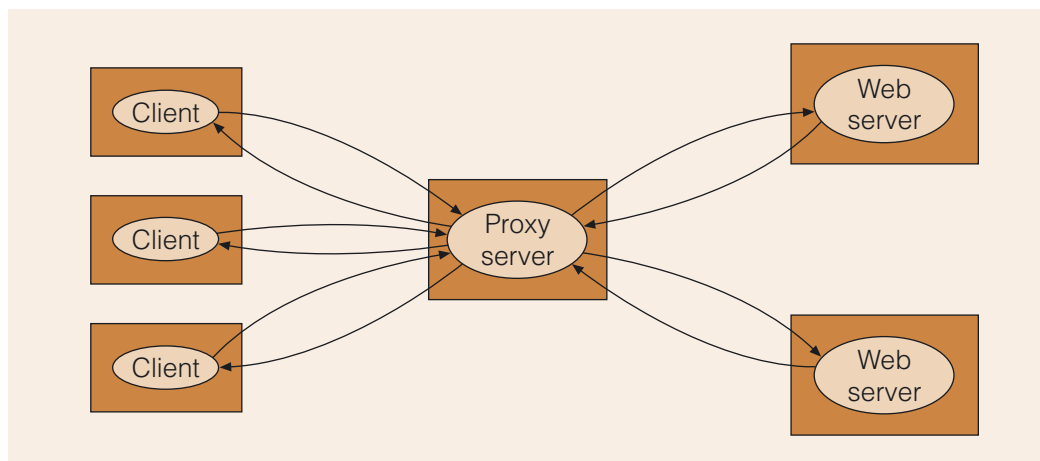


Figure 4 A proxy server

Figure 4 shows an example in which clients make requests and receive replies from the proxy server. It also shows how the proxy server makes requests and receives replies from the web servers. Usually the user on a client machine is not aware that contact is made to the proxy rather than to the server it intends to connect with.

Proxy servers can provide a number of different services.

- ▶ Proxy servers can remove identifying information from a client's request, and thus provide a form of anonymity for users. A client's IP address can be hidden when the proxy makes the request on its behalf, and this offers a form of protection, reducing some of the security risks that internet users are exposed to.
- ▶ So-called web proxies focus mainly on internet traffic and aim to manage policies for usage. Many schools employ web proxies to block offensive web content and filter the content of materials accessed through the internet.
- ▶ Most proxy servers also provide a **caching** service, which means that copies of frequently requested web pages can be stored locally and supplied to the client faster than if each client was making a new request.

Many companies and large organisations, including The Open University, use proxy servers on their websites. The caching facility of proxies provides an improvement in performance. For example, if a major news story breaks, many people in the organisation may want to be kept informed of the latest developments, and connect to an online news provider such as the BBC website. This website may get swamped with requests for information, and may have difficulty responding to all the requests. However, by caching the latest news on the proxy server the users will get the news from the proxy, without necessarily being aware that they are accessing a local copy of the data, rather than data coming directly from the server.

The study of algorithms to ensure that proxy servers hold the most up-to-date information is an ongoing, advanced, research topic.

2.4 Peer-to-peer style

Peer-to-peer is a software architecture style in which computer resources and services are directly exchanged between computer systems without the need for distinct clients and servers. In such an arrangement, computers communicate directly among themselves and can act both as clients and as servers, assuming whichever role is most efficient for the network at the time.

In the peer-to-peer style the participating processes play equal roles, with equivalent capabilities and responsibilities (hence the term 'peer'). Each participant may issue a request to another participant and receive a response. This is shown in Figure 5.

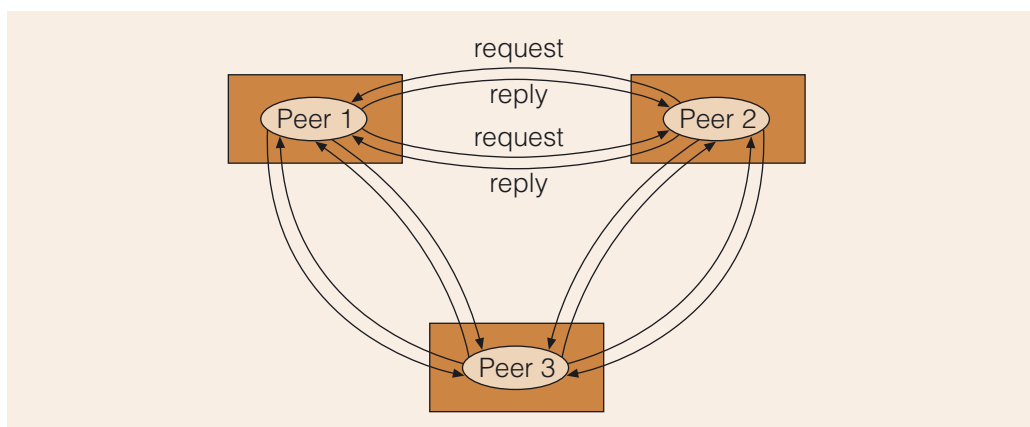


Figure 5 Peer-to-peer style

There are two main types of peer-to-peer network: pure peer-to-peer, where all participating processes are both clients and server; and hybrid peer-to-peer, where most processes are peers, but some hosts act as servers and have some central role.

Peer-to-peer networks, often abbreviated to P2P, have been in existence since the 1970s, but came to fame due to the publicity surrounding Napster, a file-sharing service for music files, which resulted in long court cases regarding copyright laws.

The Napster P2P system

Napster started in 1999, on the computer of a student who wanted to share music files with his friends. Napster used what is now known as a hybrid P2P network, because it combines the client-server model with the P2P model. The server stored an index of all the files that could be shared, a list of the users who were online and records of which user owned which file. Users could look up the files they were interested in, and have Napster make direct contact with a user who held that file.

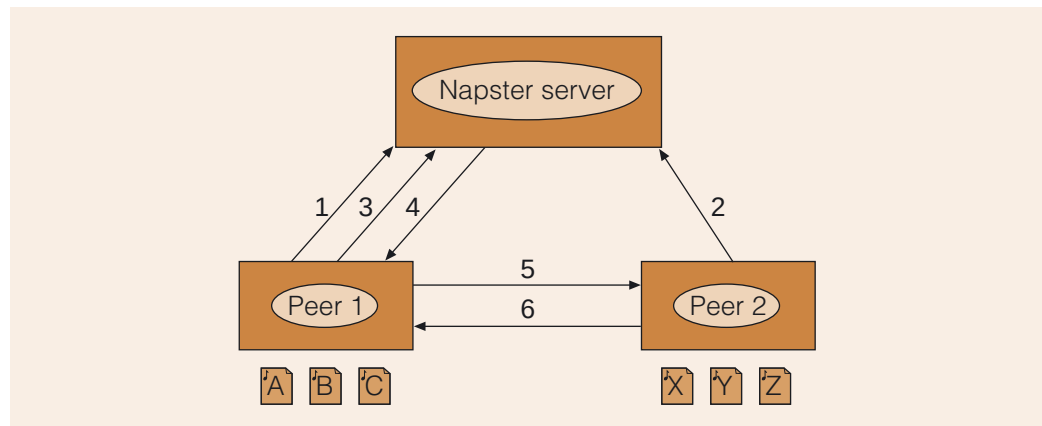


Figure 6 An example of using the Napster file server (based on Capra, 2004)

Figure 6 shows the steps that were involved in using Napster.

- 1: Peer 1 logged on to Napster and sent it a list of files (A, B and C) to be shared.
- 2: Peer 2 logged on to Napster and sent it a list of files (X, Y and Z) to be shared.
- 3: Peer 1 searched Napster for file Y.
- 4: Napster returned to Peer 1 a list of users holding file Y; in this example one such user is represented by Peer 2.
- 5: Peer 1 requested file Y from Peer 2.
- 6: Peer 2 sent file Y to Peer 1.

If a new music CD was released in Australia, for example, someone buying that CD could copy it onto their PC, and users all over the world could download it, for free, sometimes even before it was released in their own country! The music industry was worried about this development, and was successful in its court battle that led to the closure of the free Napster service in 2004.

The concept of Napster has opened the eyes of many in the music industry to a new business model based on *deliberately* sharing files with other users. For example, artists can encourage the sharing of their music, free of charge, thus building up a huge fan base on a low marketing budget.

Peer-to-peer compared to other software architectures

The peer-to-peer style offers a different solution to the problems of availability, responsiveness and failure that can occur in distributed systems. That is, in peer-to-peer networks all clients provide resources, including bandwidth (a measure of data transfer capacity) as well as storage space, and computing power. Thus, if more computers connect to the P2P network and the demand on the system increases, the total

capacity of the system also increases. This is not the case with the client–server style in which there is normally a fixed set of servers and where adding more clients means a slower rate of data transfer for all users.

When discussing the multiple server style, we noted that data replication can improve the robustness of a system, since if one computer fails for some reason, others can take over. In P2P systems, data can also be replicated over multiple peers.

Furthermore, in the case of pure P2P systems where there is not even the reliance on a centralised server to find data, there is no single point of failure in the system, which means that there is not one single point that could cause a major disruption to the service.

SAQ 1

- (a) Which factors measure the performance of a distributed system's software architecture?
- (b) What are the weaknesses of the client–server style?
- (c) What different types of services can be offered by proxy servers?
- (d) What is load balancing? Give an example of load balancing for each style of software architecture discussed.

ANSWER.....

- (a) The performance of a system can be measured by responsiveness and throughput.
 - (b) The central role of the server forms a bottleneck and thus limits the performance of the system. This central role also forms the single point of failure: if the server goes down, the whole system goes down.
 - (c) Proxy servers can provide:
 - ▶ a caching service by storing local copies of previously visited web pages on the proxy;
 - ▶ a form of security, by anonymising the IP address details of clients;
 - ▶ implementation of data content policies, by allowing acceptable content to reach a client and blocking offensive web content.
 - (d) Load balancing is a strategy to improve the performance of a distributed system, such that the workload of the system is shared among many processes, rather than carried out by one process which inevitably leads to other processes having to wait. For the system architectures discussed:
 - ▶ in the classic client–server system architecture, load balancing can take place if some of the processing is carried out by the clients;
 - ▶ in the multiple servers model, the load can be shared among the servers;
 - ▶ in the proxy server model, the proxy takes over some of the work from the servers, and relays information to the clients that would otherwise have had to be provided by the server;
 - ▶ in the peer-to-peer architecture, the work is shared among all the peers.
-

3 Tiered architectures for client–server systems

In this section you will study developments in the architecture for client–server systems in terms of increasing levels of separation of functionality. You will look at the simple two-tier approach and the three-tier (or n-tier) approach, discussing the strengths and weaknesses of each architectural model.

We return to the Music Shop case study introduced in *Unit 1*. There we saw a simple system to handle the business relating to second-hand and rare instruments. This system runs on a simple PC and customers take turns to view the data on the PC. The system consists of one program, which is responsible for everything: accessing the data, arranging it into the necessary form and presenting it to the customer.

In this unit we consider a more sophisticated scenario, in which we have multiple PCs communicating with a server which accesses a database management system (DBMS). Now one task – access to the data – is the responsibility of a program running on the server, while the rest of the work is carried out by other software running on the customer PCs and communicating with the server over a network.

Music Shop: further developments

Business is booming. The warehouse holds more and more second-hand and rare instruments. News gets around and the shop starts to become crowded with customers who want to use the PC in the shop to search the instrument catalogue. The shop manager decides that they need to have more PCs for customers to browse the catalogue. The shop wants to set up a separate ‘music café’ containing these additional PCs. This café will be set up in the basement of the building and the PCs will be connected to the server in the shop via a local area network (LAN).

The staff who work in the shop have access to the server PC and are able to keep the catalogue up to date. Furthermore, there is a computer in the warehouse to help keep track of goods coming in and going out, and of what is currently in the warehouse.

Benefit

More people can now search through the catalogue simultaneously, and make reservations. The system can potentially be expanded to accommodate many more customers.

The shop has effectively implemented what is referred to as a two-tier architecture.

3.1 Traditional two-tier model

The traditional **two-tier model** is shown in Figure 7. It is fairly simple: client applications, which may be running on PCs, access the database server. The clients, taken together, form one tier – the client tier – and the database server forms another tier – the database tier.

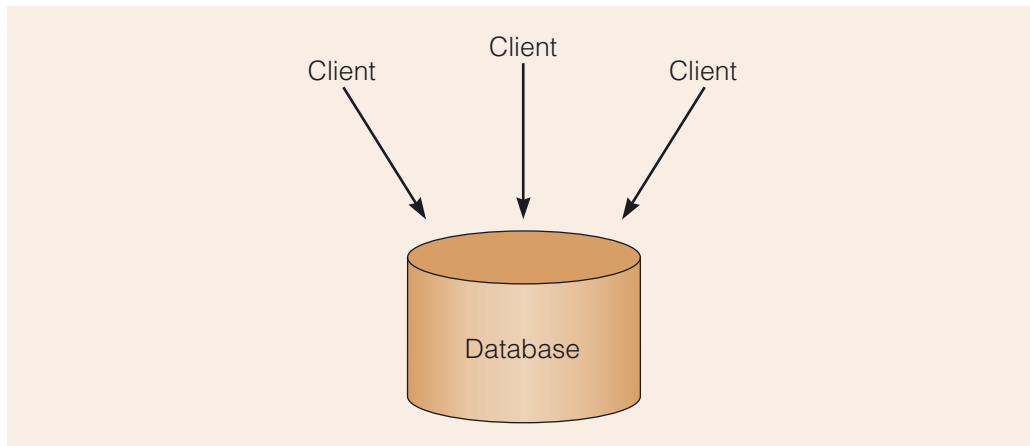


Figure 7 Traditional two-tier architecture

A **tier** denotes a common functionality at quite a high conceptual level. For example, for the client tier we make no assumptions about how such clients might be implemented, or what their software or hardware requirements are. All we say is that they 'act as clients'.

In a traditional two-tier application, the processing is carried out by the clients and not by the server, as would be the case in a centralised system. These clients are sometimes referred to as **thick clients**, in contrast to the *thin clients* we will discuss later. The database acts only as a repository for data. There are some simple constraints enforced by the database, which disallow operations that would cause obvious problems in terms of the consistency of the data.

The two-tier architecture became a possibility when PCs became powerful enough to move the programs that facilitate the display of, and interaction with, data to the client side. This has the advantage that clients can be independent of each other – that is, they may be working with different presentations of the data. For example, there may be different ways of presenting the data depending on the type of user: general users and customers are allowed to see certain sections of data, whereas privileged users, such as the administrators in charge of maintaining the data, are given a different view.

There are a number of applications that are well suited to a two-tier architecture: for example, where the number of users of the application is expected to be limited, or if the data is not overly sensitive and thus not requiring high levels of security.

The two-tier architecture also has its disadvantages.

- ▶ **Scalability.** This is the ability of a system to scale to support larger or smaller volumes of data and more or fewer users. The two-tier architecture does not scale well, because the performance of applications can be expected to degrade rapidly when the number of concurrent users reaches a certain threshold. This is because the maximum number of clients that can interact with the server is set by the number of connections that the server supports. Furthermore, if a client creates a particularly heavy workload for the server, this will directly affect the work of other clients, since they are all competing for the same resource.
- ▶ Poor strategy to deal with failure. If the server fails, then none of the clients will be able to work.

- Application distribution and remote usage. If changes are made to the software for the client then the new version will need to be distributed to each client. When there are many users, this may entail a considerable administrative overhead. Moreover, users may be unaware of the existence of the new version, or may be reluctant to go to the trouble of downloading and installing it!

3.2 Layered architectures: interfaces and components

In the literature the terms 'tier' and 'layer' are often used interchangeably, but throughout this course we will make a distinction between the two concepts. We will use the term tier as defined above, namely to describe a shared functionality at a high conceptual level. The term **layer** we reserve for the idea of a layer of software in which implementation details are hidden, but where the functionality of the layer can be invoked through a well-defined interface. In this subsection we give examples of such layered approaches.

The Open Systems Interconnection reference model

A well-known software model that uses this layered approach is the Open Systems Interconnection (OSI) reference model developed by the International Standards Organization (ISO). This model consists of seven layers as shown in Figure 8.

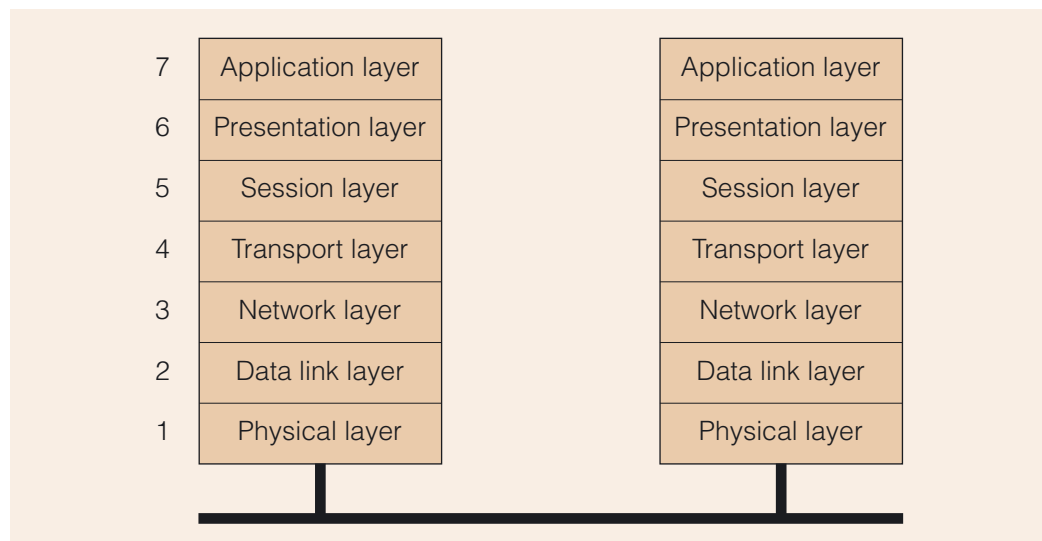


Figure 8 The OSI seven-layer model

The **OSI model** defines a modular approach to networking, with each layer responsible for some discrete aspect of the networking process. The model describes the flow of data in a network, from the lowest layer (the physical connections) up to the layer that directly interfaces with the user's applications.

Data going from a particular computer to the network is passed from layer to layer. Each layer is able to communicate with the layer immediately above it and the layer immediately below it. Thus each layer is written as an efficient, streamlined software component.

The highest layer, the application layer, performs common application services for the application processes. For example, it defines a file transfer protocol (FTP). At the lowest level is the physical layer, which is concerned with transmitting bits across a communication channel from one computer to another.

Figure 9 shows how a typical operating system maps onto the OSI model, as well as a number of protocols that you will have come across.

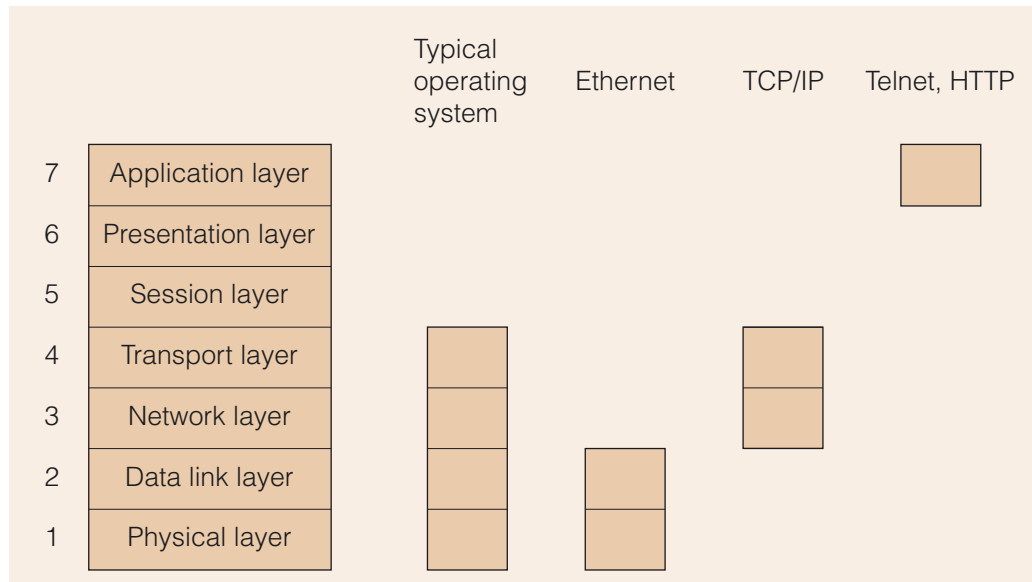


Figure 9 Mapping an operating system and protocols onto the OSI model

For example, most operating systems span the layers from the physical layer to the transport layer, where the physical layer provides an interface between the actual physical network and the operating system, and the session layer provides an interface for system calls for network communication. The Ethernet standard, briefly described in *Unit 1*, operates at the two lower levels of the OSI model, the TCP/IP protocol operates at layers 3 and 4, and HTTP and Telnet operate at layer 7.

James Bond and the OSI model

The seven-layer model is sometimes humorously extended to refer to non-technical issues or problems. One common joke is to refer to the *ten-layer model*, with layers 8, 9 and 10 being the 'user', 'financial' and 'political' layers.

Dick Lewis uses the analogy of a message carried by James Bond, the fictional British agent, famous from more than twenty books and films since the 1970s, to explain the OSI reference model. The functionality of each layer is explained as follows.

James Bond meets Number One on the 7th floor of the spy headquarters building. Number One gives Bond a secret message that must get through to the US Embassy across town.

Bond proceeds to the 6th floor where the message is translated into an intermediary language, encrypted and miniaturized.

Bond takes the elevator to the 5th floor where Security checks the message to be sure it is all there and puts some checkpoints in the message so his counterpart at the US end can be sure he's got the whole message.

On the 4th floor the message is analyzed to see if it can be combined with some other small messages that need to go to the US end. Also if the message was very large it might be broken into several small packages so other spies can take it and have it reassembled on the other end.

The 3rd floor personnel check the address on the message and determine who the addressee is and advise Bond of the fastest route to the Embassy.

On the 2nd floor the message is put into a special courier pouch (packet). It contains the message, the sender and destination ID. It also warns the recipient if other pieces are still coming.

Bond proceeds to the 1st floor where Q has prepared the Aston Martin for the trip to the Embassy.

Bond departs for the US Embassy with the secret packet in hand. On the other end the process is reversed. Bond proceeds from floor to floor where the message is decoded.

The US Ambassador is very grateful the message got through safely.

'Bond, please tell Number One I'll be glad to meet him for dinner tonight.'

(Lewis, 2007)

Interfaces and components

The layered approach is also used as a way of building large software systems, by putting together software components. A **software component** is generally defined as an independent software unit that can be composed with other components to create a software system. Examples of components that you will have come across already are operating systems, relational databases and web browsers. During this course you will also learn about components such as servlets and various Enterprise JavaBeans (EJB) components (we will discuss this further in Section 4).

Each component has an **interface**, which defines the communication boundary between two components. An interface generally refers to an abstraction that a component or some other entity provides of itself to the outside. Through the interface the entity presents methods for external communication, and the interface allows the entity to be internally modified without affecting the way outside entities interact with it.

The two-tier architecture forced designers of distributed systems to think in terms of published interfaces. In order to be able to develop clients, the server had to have a known, stable interface. Hence this architecture is also associated with a key development in software for distributed systems: the concept of the **application programming interface (API)**. The API defines how to invoke a service and the responses that can be expected. Once servers had a well-known API, different sorts of client could be developed. Also, as long as the API is kept the same, the server software can be modified internally without affecting the clients.

3.3 Musical instruments

We now return to the case study of the Music Shop, and look at some of the developments to the software system's architecture.

The Music Store Internet Café

Up until now, the café, with its PCs to access the database, had been in the same premises as the shop. However, the number of people visiting the café is increasing, and the shop is considering moving the café to premises across the road. As the staff are planning the move of the café, they sit down to rethink the design of their system.

From talking to their current customers they learn that quite a few of them are Apple Mac users rather than PC users. Some of these customers suggest that the store should put some Apple Macs in the café, as this is likely to attract additional users.

Introducing Macs into the café will have consequences. With the current setup, the software running on the customer PCs has a lot of the business logic mixed in with the logic of the user interface. For example, the rules governing the delivery charges are currently implemented in the part of the program that creates the GUI to display the total cost of the item purchased. If they now have to create a GUI that is suitable for Macs it would be better if the business rules could be stripped out. This would avoid having to repeat the same business rules in two different places, with the attendant problem of ensuring that any changes in one place were replicated in the other place.

They also realise that if they are opening up the café across the road, they could use the internet for the communication between the server and the clients. This would allow them to go online completely, thus increasing their number of potential customers enormously. At the moment the applications for customers to interact with the system are developed in such a way that they can be installed and run on the PCs in the café. However, to enable customers to interact with the system through the internet from anywhere in the world, they need to develop a web version. Again this would plead for separating the business functionality from the presentation layer.

At this point the Music Shop adopts the three-tier, or n-tier, architecture for its computer system.

3.4 The n-tier architecture

Software systems with three or more tiers are often called n-tier applications (where n stands for the number of tiers). These n-tier applications strive to make a clear distinction between the separate operations of the application. Large n-tier applications will often be constructed of distributed objects spread across many machines in multiple physical locations. In this brief discussion we will restrict ourselves to describing a typical three-tier architecture, but more complex systems with more tiers are similar in concept. We describe an example of such a system in Section 4.

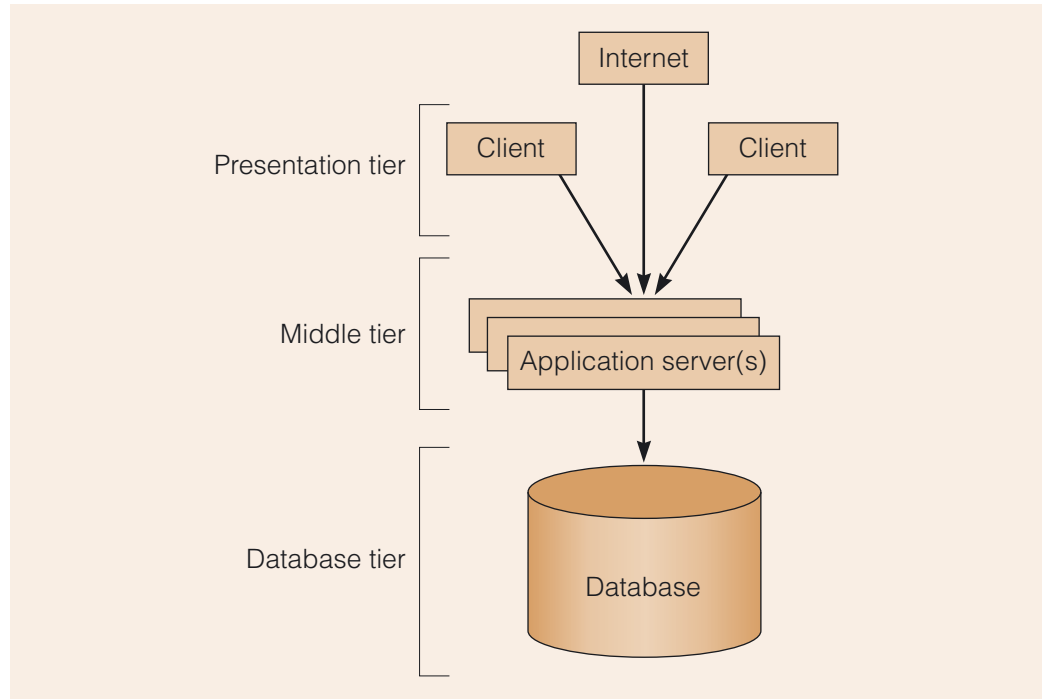


Figure 10 The n-tier architecture

Figure 10 shows the three tiers of a typical example of this architecture, each with its own functionality as follows.

Database tier

In *Unit 6* we discuss the functionality of the DBMS.

The database tier contains the Database Management System (DBMS) and the data, and is generally for the storage of data. The database can be placed in a single location or it can be distributed. In Subsection 2.2 we discussed the possible importance of keeping data replicated in several databases, or having data partitioned in such a way that each location stores a certain subsection of the data. The advantages to having the database distributed can be improved fault tolerance and better performance.

Middle tier

The middle tier consists of one or more application servers. An **application server** is, as the name suggests, a server dedicated to running a number of applications, as opposed to a file server which stores files, or a mail server which deals exclusively with mail.

The middle tier contains the software processes to handle the **business logic**, which is generally defined as the calculations and processes appropriate for the business. This may refer to methods for data entry and data querying or report processing, or it may refer to software to implement business rules such as those relating to discount rates, delivery charges, or different types of membership for the organisation.

In this middle tier the bulk of the processing takes place and, if necessary, more application servers can be added to support increased levels of activity. This may be done such that each application server deals with a particular function, for example one may deal with the selling of products and another one with the buying in of new products for the business. Or it may be that several application servers handle the same part of the functionality, but can share this processing between them.

Presentation tier

The presentation tier comprises all the software processes to present a GUI, or indeed a non-graphical interface, to the user. The tier consists of a group of hosts, all of which may have different types of software running, such as applets, 'standard' applications, or internet clients running within a browser. As a group, these different hosts perform an equivalent role as far as the overall system is concerned – they deal with the presentation of the application.

Since the business logic has been extracted from this tier, the processing in this tier is relatively 'light' for the clients. We say that the clients in this architecture are **thin clients** compared with the clients in the two-tier architecture.

Benefits of the n-tier model

- ▶ *Scalability.* This is the key benefit of the n-tier architecture because it is easy to add more application server machines if the number of users or the amount of data increases significantly. Also, the database requires connections from only a small number of application servers, rather than from every client, thus avoiding the constraint which determines the upper limit of the scaling of two-tier systems.
- ▶ *Reuse.* In essence, the n-tier architecture is a form of modularisation, such that each tier has its own functionality. Part of the philosophy behind modularisation is that it improves the reuse possibilities of software. For example, if a lot of effort has been spent on developing an interface suitable for a group of users with very specific needs, say visually impaired users, then ideally we would want to be able to reuse such an interface for other applications. If the business logic is separate from the presentation it is more likely that such an interface can be reused for an application in a different domain and with different business logic.
- ▶ *Improved data integrity and security.* Since all updates go through the middle tier, the middle tier can ensure that only valid data is allowed to be entered into the database. At the same time, security can be improved by implementing it at multiple points (not just at the database). Since the client does not have direct access to the database, it is more difficult for a client to obtain unauthorised data. (Unfortunately, there are also security drawbacks – see below.)
- ▶ *Improved availability.* Redundancy is an important feature in this architecture. That is, by adding more application servers than seems strictly necessary (which is very easy to do), the system allows, even *plans* for, the possibility of the failure of one or more servers. In such cases the system can make use of redundant application servers so that it can recover from network or server failures.

Modularity is an important concept in software engineering and programming languages. Modularity is the property that measures the extent to which computer programs have been composed of separate parts, called modules. Procedural programming and object-oriented programming both strive for modularisation.

Drawbacks of the n-tier model

- ▶ *Increased complexity/effort.* In general, it is more difficult to build a three-tier or n-tier application than a two-tier application. One complicating factor is the doubling of the number of points of communication: a single communication between client and server is replaced by two – from client to middle tier, and from middle tier to database.
- ▶ *Security.* Given that there are now more points in the system needing to be secured, there are additional security risks.

SAQ 2

- (a) Give the advantages and disadvantages for the two-tier and n-tier architectures.
- (b) What are thin and thick clients? Explain which type of client is more likely to be used in the two- and n-tier architectures.

ANSWER.....

- (a) Advantages of both two-tier and n-tier: can incorporate separate presentations for different types of user.

Disadvantage of two-tier: does not scale well, poor strategy for dealing with failure, difficulty in updating client software for remote users.

Advantages of n-tier compared to two-tier: better scalability, encourages reuse, improved security and availability.

Disadvantage of n-tier: increased complexity, increased security risks.

- (b) Thin clients (typically used in n-tier architectures) do not carry out much processing and leave this to the server. Thick clients (more regularly used in two-tier architectures) carry out more processing.
-

4

Java Enterprise Edition

In *Unit 4* we discussed how the Java language was developed to be platform independent and the role of the Java Virtual Machine (JVM) in attempting to achieve this. It is not easy to develop a language that will work equally well on both a supercomputer and a small device embedded in a washing machine. In *Unit 4* we discussed that different platforms have different capabilities in terms of memory size, processor speed and GUI, and that therefore Java technology has been divided into editions. In this section we will give an overview of Java Enterprise Edition (Java EE).

We will begin with a discussion of the developments in The Music Store Internet Café, and then discuss what is meant by an enterprise and why such organisations might have specific software and technology needs.

From café to enterprise ...

The internet café across the road is a great success, and has built up quite a reputation among musicians. The fact that the shop has opened up the catalogue browsing facility to the internet has meant that a lot more customers are now aware of the existence of the shop and this has led to an increase in the amount of business. The staff in charge of the café side of the operation see great potential for further expansion.

Up until now customers could only browse the catalogue and make reservations for items. The reservations would hold for only a limited time, and customers had to get in touch with the shop by telephone or in person if they wanted to purchase an item. However, the concept of online shopping is becoming more common and the management decide to give this a try. The new system now has to deal with payments.

The Music Store server has to deal with catalogue searching, reservations and payments. The server also has to communicate with various financial systems in order to handle payments.

Furthermore, the warehouse processes must be automated, if only to help cope with the increased scale of the business. There will be a separate server in the warehouse which will communicate with the shop server, but it will not be publicly accessible. The warehouse server will log all incoming and outgoing items, and will also deal with the processing of data relating to the delivery of items to customers.

Customers register with the system before use and enter their contact details. Once they wish to purchase an item they need to give their financial details and an address for delivery.

Benefits

- ▶ The system can be used by many customers wherever they are and whatever the time of day.
- ▶ The warehouse staff can record dispatches and returned items and the catalogue can be automatically updated to reflect these changes.

Problems

- ▶ There are potential security problems, now that the system has to deal with customer details and payments.
- ▶ The shop system or its communication links could become overloaded if too many people want to use the system at once. There may be a need to have several (or many) servers to deal with high demand.

As the management begin to think their way into the new online system, they identify potential different ways to expand the business even further. In particular, they envisage that in the near future they would like to go into online auctioning. They have noticed that some very rare instruments are difficult to sell from the shop, but might benefit from an auctioning business model.

This new enterprise approach gets everyone excited, and they decide that if they want to go really, really big, they will need to make sure that their software infrastructure is robust and expandable.

4.1 What is an enterprise?

The term enterprise refers to an organisation, which may be small, medium or large, and non-profit, educational or commercial. However, one of the main characteristics of an enterprise is that it wants to *expand*.

An enterprise has different types of resource that help it achieve its goals, and that are of value to the enterprise:

- ▶ physical resources – these may be the office buildings, the products it is selling or making, the books it uses in its teaching, or the laboratory equipment necessary for its research;
- ▶ human resources – the employees, students, customers or partners;
- ▶ information and knowledge – these may be customer details, including these customers' bank account details, the recipe of its award-winning soup, or the documentation for its software system.

The above resources are very important to the success of the enterprise, and the enterprise needs to be able to protect them. It is also very important that the size of the various resources can increase, in order for the enterprise to be able to expand.

Modern enterprises pursue the use of information technology in order to build enterprise systems that help them effectively foster their goals and objectives.

- ▶ An enterprise would normally be interested in extending its customer base, and it therefore has a need for internet connectivity, as we saw, for example, in the Music Store Internet Café. At the same time, it must be able to preserve its existing customer base, and this may require being able to use **legacy systems**. Enterprises may also be interested in technologies for user interfaces, because being able to tailor the interface of their computer systems for different types of user can give them a business advantage.
- ▶ In order to be able to *preserve* existing resources, and to enable their *growth*, an enterprise would try to *integrate* the legacy software systems with modern, distributed applications. Another form of integration that can help expand the information resource in, for example, a hospital or a medical research laboratory, is to have good connectivity to embedded devices, such as medical diagnostic devices.

A legacy system is a computer system or application program that continues to be used because the cost of replacing or redesigning it is considered to be too high. This is often despite its poor competitiveness and poor compatibility with modern equivalents. The implication is that the system is large, monolithic and difficult to modify.

- ▶ The *protection* of resources can be established through security measures. The *accessibility* of resources can be enabled via communication technologies that connect resources to the enterprise.
- ▶ The growth and preservation of information can be managed in a database system. If the enterprise is keen to make more information available to its users or customers then, again, this may call for web connectivity.

See *Unit 10* where we study security.

The above points will give you an insight into how a modern enterprise, in its quest for expansion, will turn to various forms of technology in order to pursue its objectives. In the next section we discuss how some of the technologies that modern enterprises pursue have been brought together in the Java Enterprise Edition (Java EE).

4.2 The Java Platform, Enterprise Edition

The Java Platform, Enterprise Edition (Java EE) is a software framework that brings together a number of technologies used in enterprise systems. In particular, a lot of effort was spent on straightening out the issues in the middle tier of the three-tier architecture, splitting it into two separate tiers: the web tier and the business tier. This is shown in Figure 11.

In 1999 Sun Microsystems came out with the J2EE platform for building enterprise systems and, in 2006, this was updated to Java EE 5.

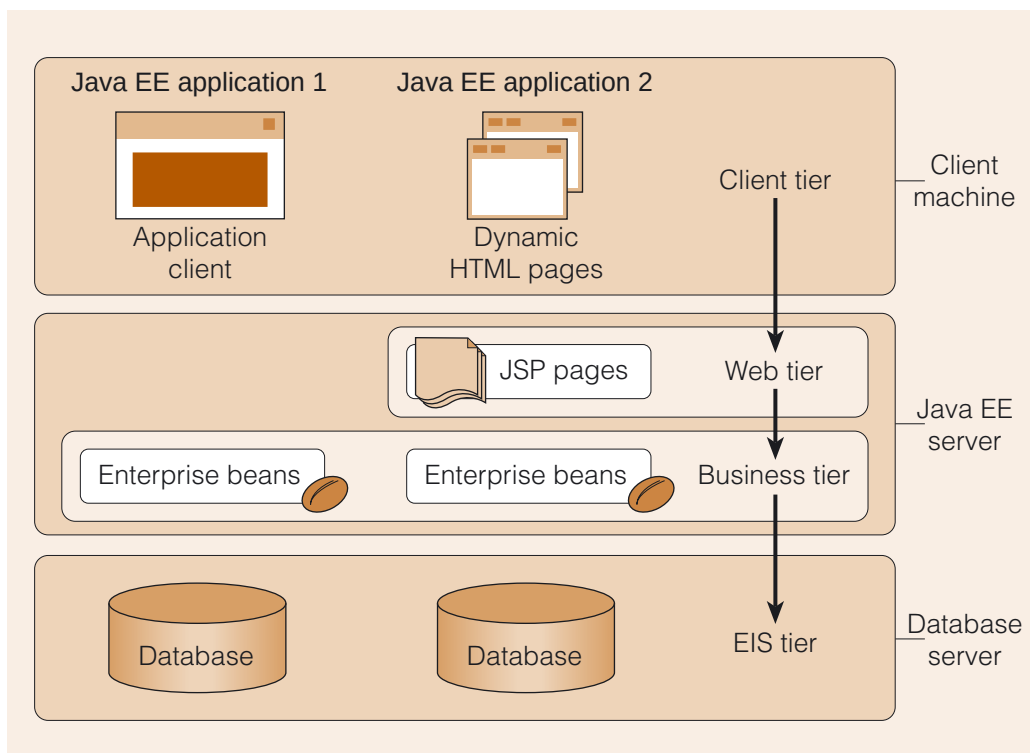


Figure 11 The tiers of the Java EE 5 framework (based on Sun Microsystems, 2007)

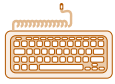
A Java EE application can consist of either three or four tiers, as shown in Figure 11. The purpose of each tier is as follows.

- ▶ The *EIS tier* (the enterprise information system tier) holds the data for the application and stores it permanently in some kind of database. As was the case with the database tier in the n-tier architecture, this tier can be located on one machine or spread over several machines. It is also possible that this tier contains some form of legacy information system.
- ▶ The *business tier* contains the software relating to the business logic.

- ▶ The *web tier* is the tier that is required in those cases where the clients are web clients. Its role is to make the business logic accessible for web applications.
- ▶ The *client tier* deals with the presentation, or user interface, of the application as a whole. The clients can be in many different forms: applets, web clients (such as a web browser) or application clients. An application client can interact directly with the business tier; in the case of web clients we need to use the web tier.

The two middle tiers, the web tier and the business tier, are shown with what are known as Java EE components: JavaServer Pages (JSP) and Enterprise JavaBeans (EJB). You will learn more about what these and the other components are intended to do and how they work in *Units 6 to 9*. Here we want to draw your attention to the fact that these components in the two middle tiers are designed to be run on a Java EE server. But what does that mean?

Recall that in Subsection 3.2 we defined components as independent software units that can be composed to create a software system. Java EE components are written in the Java programming language and are compiled in the same way as any program in the language. However, in order to *run* such a component, it must be assembled into a Java EE application and is then handed to the Java EE server program, which will run and manage the component from that point onwards.



Activity 5.1

Running the application server.

The components for the Java EE platform must strictly comply with the API which it defines. Only if a component fulfils this condition can it be made to run.

Component models

The Java EE framework is an example of a **component model** (a component model being a definition of standards for the implementation, documentation and deployment of components). Examples of other component models you will learn about are EJB (*Unit 7*) and CORBA (*Unit 11*).

A component model gives the developers of components a standard to which they should work, such as clear guidelines on how the documentation is supposed to be organised. The users of components, or the composers of components, are also helped by these standards when putting the components together.

An important idea behind component models is that they ensure that the components can interoperate. For example, when more and more developers started using Java to build enterprise systems, numerous Java APIs entered the market. However, the APIs developed by different groups were not always compatible with each other, and it was not always clear how they were supposed to relate to each other. Bringing all these components together, clarifying how such components can be composed, was essentially what the Java Enterprise Edition set out to do, thus becoming an environment for building such large enterprise systems.

Components: deployment and configuration

Components, and reusable modules generally, make it possible to divide the application development and the deployment process into distinct roles so that different people or companies can be responsible for different parts of the process.

Deployment is the step of placing or installing software on the target systems. This will include configuration steps if necessary.

When components are put together and integrated with other parts of the software system, parts of the system need to be configured to the specific needs of the organisation. This is usually carried out by a consultant who has in-depth knowledge of the component to be configured and who will be working for the vendor of the component. This consultant must work alongside the system's engineer, and gather

detailed information about the customer's business and business processes. This knowledge is then stored in specific configuration files. The task of configuring a system requires specific skills in terms of the configuration notations and the various tools used for these systems.

As more and more systems are put together from existing components, the role of the application deployer will increase in importance.

SAQ 3

What do the standard three-tier architecture and the Java EE tiered architecture have in common, and how do they differ?

ANSWER.....

The Java EE tiered architecture is a specific example of the n-tier architecture, with separate database, middle and client tiers. It differs from the three-tier architecture in that the middle tier is split into further tiers: the web tier and the business tier.

The Java EE framework in M362

In the following units of this course we will take you through each of the tiers of the Java EE framework, explaining the main issues and technologies in that tier. You will also study the particular communication issues pertaining to that tier.

Unit 6 The database tier: Handling transactions and concurrency.

In this unit you will study the issues involved in the storage of data, and how to update and retrieve data from a resource in a safe way. We look at the concurrency issues for such systems and also how to make a connection between a database and the Java applications above it.

Unit 7 The business tier.

In this unit you will study Enterprise JavaBeans (EJB), which are the components that help to model the business logic of the application. You will also study RMI as a technique for enabling distributed objects to communicate with each other.

Unit 8 The web tier.

The two main technologies studied in this unit are servlets and JavaServer Pages (JSP). Servlets are Java programming language classes that *dynamically* process requests and construct responses, whereas JSP pages are a better approach when creating *static* content.

Unit 9 The client tier.

In this unit you will study in more detail the different types of client: web clients, applets and application clients. The web clients will connect to the JSP pages in the web tier. The application clients are slightly different, and may, for example, be used in situations where a more complex graphical interface is required than would be possible using web technologies.

5

Distributed communications – message passing

In the previous sections you studied the main architectures for distributed systems. The communication between the parts of a distributed system is another important topic.

In this section you will look at some of the low-level operations that enable processes to communicate, and distinguish between some basic forms of communication. We will concern ourselves with the following questions.

- ▶ Do processes have to be *ready* for each other in order to be able to communicate?
- ▶ How does one process *address* another process in order to be able to communicate?
- ▶ What do we need to do with the *data* that may get exchanged during the communication?

5.1 Synchronous and asynchronous communication

Note that this form of message passing is entirely different from the idea of objects in object-oriented programs sending messages to each other. In this course we will use the idea of sending messages in the true distributed sense of sending physical data, and not as a metaphor for objects invoking methods.

The basic form of communication in distributed systems is message passing. One process acts as the sender, while the other is the receiver. Upon receiving the message, the receiver may or may not be in a position to pass a message back, depending on the circumstances.

The message-passing mechanism must be able to support both the actual transfer of information and the synchronisation of the processes involved. We can distinguish between asynchronous and synchronous communication.

In **asynchronous message passing**, the sending and receiving processes do not have to be at their synchronisation points at the same time. It is possible that one process is ahead of the other. If the receiving process is ahead of the sending process, it will have to wait. This is illustrated in Figure 12, scenario (a), where the dotted line for process A indicates that, having executed a *receive*, it is having to wait.

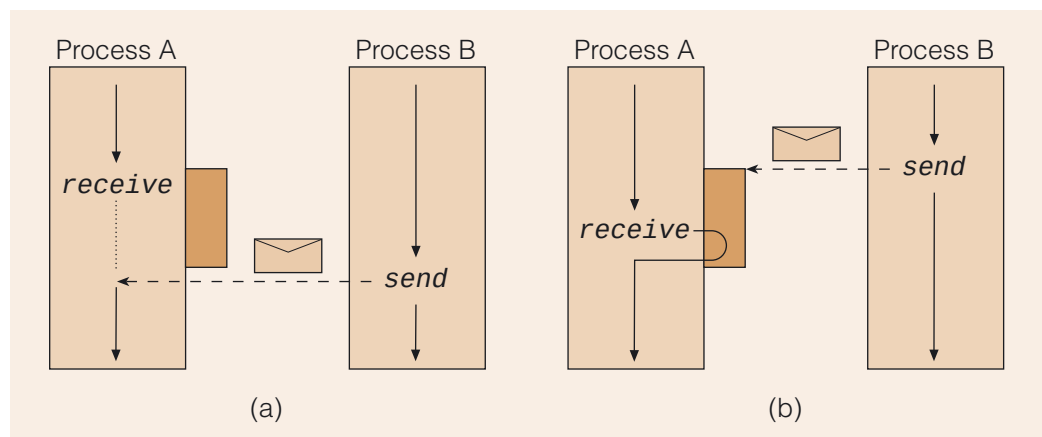


Figure 12 Two scenarios for asynchronous message passing

In Figure 12, scenario (b), we see that the sending process is ahead of the receiving process. In that case the sending process will deposit the message in a buffer, and continue processing. It will not experience any delay. The receiving process can retrieve the message from the buffer when it gets to its synchronisation point and executes a *receive*.

Thus the general requirement for using asynchronous communication is that there is a buffer, i.e. some form of data structure to temporarily store messages. The main characteristic for asynchronous communications is that the sending process does not get delayed because of the *send*, i.e. it is not blocked. It is always in a position to deposit its message and continue processing (though it may be delayed for some other reason not related to the *send*).

By **synchronous message passing** we mean that both processes are required to be at the point of sending and receiving at the same time, and the message goes straight from the sending process to the receiving process. If the receiving process is ready to receive a message, but the sending process is not ready to send, the receiving process will be delayed. Alternatively, if the sending process is ready to send but there is no process waiting to receive the message, the sending process is delayed. This second scenario is illustrated in Figure 13.

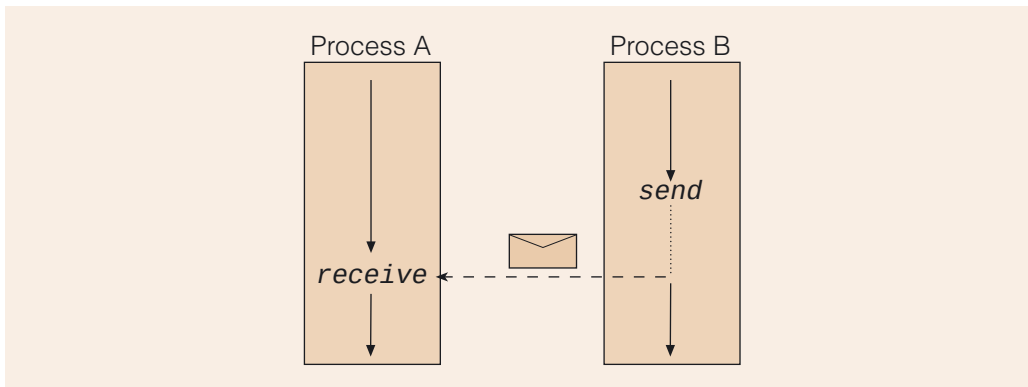


Figure 13 Synchronous communication

Synchronous message passing is also referred to as a **blocking** form of communication. The process that sends is blocked while it is in the *send* instruction. This is similar to ordinary method invocation, where a process is blocked for the duration of the method invocation.

Note that the blocking and unblocking is carried out by the operating system, through the inter-process communication (IPC) facilities, and it is not up to the programmer to block or unblock. However, even though this is all taken care of at a lower level, the programmer should make sure that they are aware as to whether or not a blocking form of communication is being used.

Threading and non-blocking *receive*

In asynchronous communication there is no blocking on the sending of messages. However, for the receiving there is both a blocking and a non-blocking variant. In the blocking variant the receiving process is delayed. This was illustrated in Figure 12. For a non-blocking *receive* variant the receiving process is able to continue processing after it has issued a *receive* instruction, and the buffer would be filled as a background activity. The receiving process then has to be notified when the buffer is filled, so that it can retrieve the message.

Obviously there are advantages to non-blocking forms of communication, as there is less delay involved, and it appears more efficient. In a system that uses Java, it is relatively easy to implement the non-blocking variant for receiving, by means of creating additional threads. That is, a receiving process can create a separate thread to handle the *receive* instruction, while the main thread continues with processing. At the moment that the message is actually required, i.e. the synchronisation point, the main thread and the temporary thread would be joined again. This is illustrated in Figure 14.

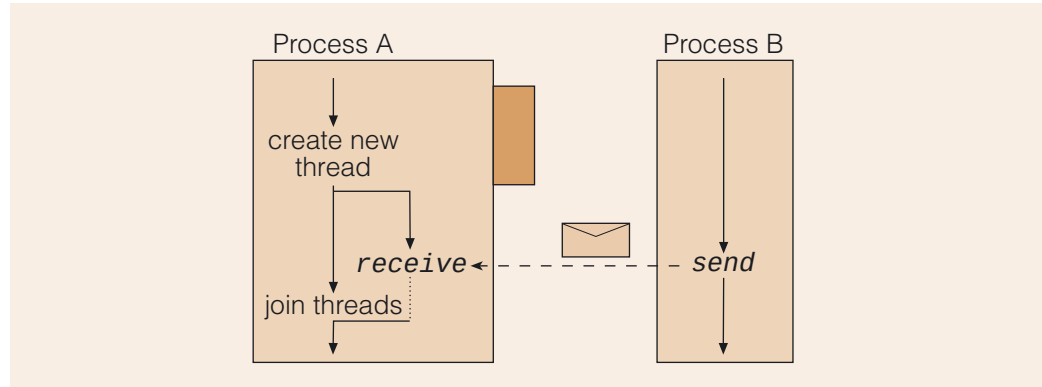


Figure 14 Multiple threads to enable a non-blocking *receive*

Even though the non-blocking form of communication appears more efficient, and can be implemented as shown with a separate thread, it is not commonly used. Programs become more complex if the main thread has to retrieve an incoming message from a separate thread.

5.2 Timeouts and deadlock

Using the blocking (synchronous) form of message passing can, in some cases, lead to deadlock. Imagine the following scenario. A browser tries to connect to a web server, but no confirmation is received from the web server that the connection was successful. This may be caused by a problem with the server or the lines of communication to the server. As a result, the browser process remains blocked, or 'hanging', indefinitely.

Obviously, the above scenario is a problem that needs addressing. The most common form of recovering from such an unsuccessful attempt to make a connection is to use what is called a **timeout**. That is, if, after a specified amount of time, the connection was not successful, it is abandoned.

A process can also hang indefinitely due to deadlock problems. In *Unit 3* we mainly discussed the possibility of deadlock arising from the locking of objects in a concurrent context. However, deadlock can also happen simply due to errors, or misunderstandings. This is illustrated in Figure 15.

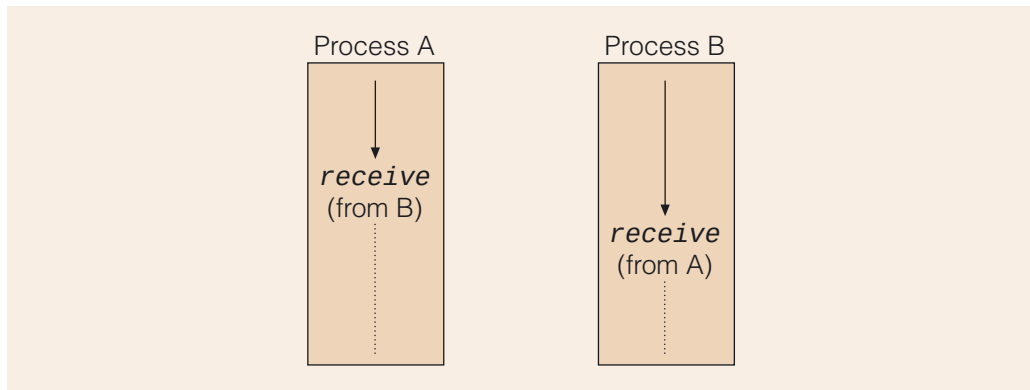


Figure 15 Deadlock and synchronous operations

Figure 15 illustrates a misunderstanding about the role of each process, or about the order of events taking place between the processes. Process A has executed a synchronous *receive* operation, waiting for input from process B, while process B has also issued the synchronous operation *receive*, waiting for input from process A. As a result, both processes are hanging indefinitely in the execution of synchronous *receive* operations, waiting for input from the other process, which can never occur. Again the above scenario could be resolved quite simply by using a timeout, thus preventing the indefinite hanging of the two processes.

5.3 Addressing

In order to enable communication between processes in a distributed system, it is a requirement that the processes involved are able to identify each other. Hence there must be a clear and standard way of naming processes. This is referred to as **addressing**.

In internet protocols, messages are sent to addresses which consist of the pair (internet address, port). The first part, the internet address, refers to a specific computer known to the internet, and there are different naming schemes for this. All computers have a unique code, known as their **IP address** (Internet Protocol address), which is essentially numeric, or we can use the symbolic address of a computer.

For example, the symbolic address of the Open University web server is `http://www.open.ac.uk` and the corresponding numeric IP address is `194.66.152.28`. In order for communication to be possible, the symbolic address has to be translated, and the actual computer located. This is done through a name service, known as the internet Domain Name Service (DNS) which is a key part of the internet.

The other part needed to address a process is known as a **port**. A port is an abstraction of a physical place on a computer through which communication can proceed between processes, and is specified as a number. Note that in this context, a port is not a piece of hardware (such as a USB port), but rather it is a logical idea of a destination. Ports are useful in order to map incoming data to a particular process running on a computer.

Examples of such a pair could be:

```
(www.open.ac.uk, 4261)      // symbolic addressing
(194.66.152.28, 4261)     // numeric addressing
```

Operating systems have processes assigned to specific ports, which are continuously listening for special types of message. Ports in the range 0 to 1023 are known as the 'well-known ports' and are reserved for special purposes – for example, connecting

to port 13 will return the date and time of the computer, and port 21 is always used for FTP requests. Ports with numbers in the range 1024 to 49151 are so-called 'registered ports', whereas ports with numbers in the range 49152 to 65535 are free and can be used by any application.

Exercise 1

Use the internet to find the following information:

- Which port number is used for HTTP requests?
- What is port number 443 used for?

Discussion

A good place to look for this information is the website of the Internet Assigned Numbers Authority (IANA).

- Port number 80 is used for HTTP, to make internet connections.
- Port number 443 is used by the HTTPS protocol, a variant of the HTTP protocol which runs over a secure socket layer, and is a more secure form of HTTP. We will return to this topic in *Unit 10*.



The Internet Assigned Numbers Authority.

5.4 Data in message passing

The message that is being passed around during message passing is essentially a piece of data. The data can be in many different formats at the level of the application (i.e. the top layer in the OSI seven-layer model we saw in Figure 8): it may be an integer, a string or an array of objects. However, at the moment when the data is actually being sent, it has to be in the form of a binary stream. Therefore, before a message is sent, the operating system's IPC facility converts the data into bits, and, upon arrival of the bits, the bits are converted back into the appropriate data type.

This sounds simple enough, however there are a number of issues related to how to deal with these different formats. The first is to do with the **heterogeneity** of the hosts, that is, the processes involved in the message passing may be running on machines that use different data representations. For example, one machine may use 16 bits to represent integers, and another machine may use 32 bits. Hence there needs to be an agreed way to resolve the discrepancy between the two formats. There are three strategies to resolve this.

- ▶ The sender converts the data into the representation of the receiver before sending.
- ▶ The sender sends the data in its own representation, and the receiving process converts it to their representation on their machine.
- ▶ Both parties agree on an external representation. The data will be sent using the external representation, and on receiving the receiver can convert it from the external representation to the local presentation.

The second issue is how to deal with more complicated data structures, such as an array of integers or a list of data structures, where each data structure may contain a number of other types. Not only do we need an external representation of the values held by these structures, but we now also require a mechanism to '*flatten*' the structures (i.e. convert the complex data structure into a flat structure such as a stream of bits), and to '*unflatten*' them on arrival (i.e. convert the flat structure back into the original data structure).

A binary stream is a stream of bits (0 and 1) such as 001001 ... 1100111.

The term **marshalling** is used to refer to the process of taking a collection of data items and assembling them into a form suitable for transmission in a message. Conversely, **unmarshalling** is the process of disassembling them on arrival in order to produce an equivalent collection of data items at the destination. These processes are illustrated Figure 16.

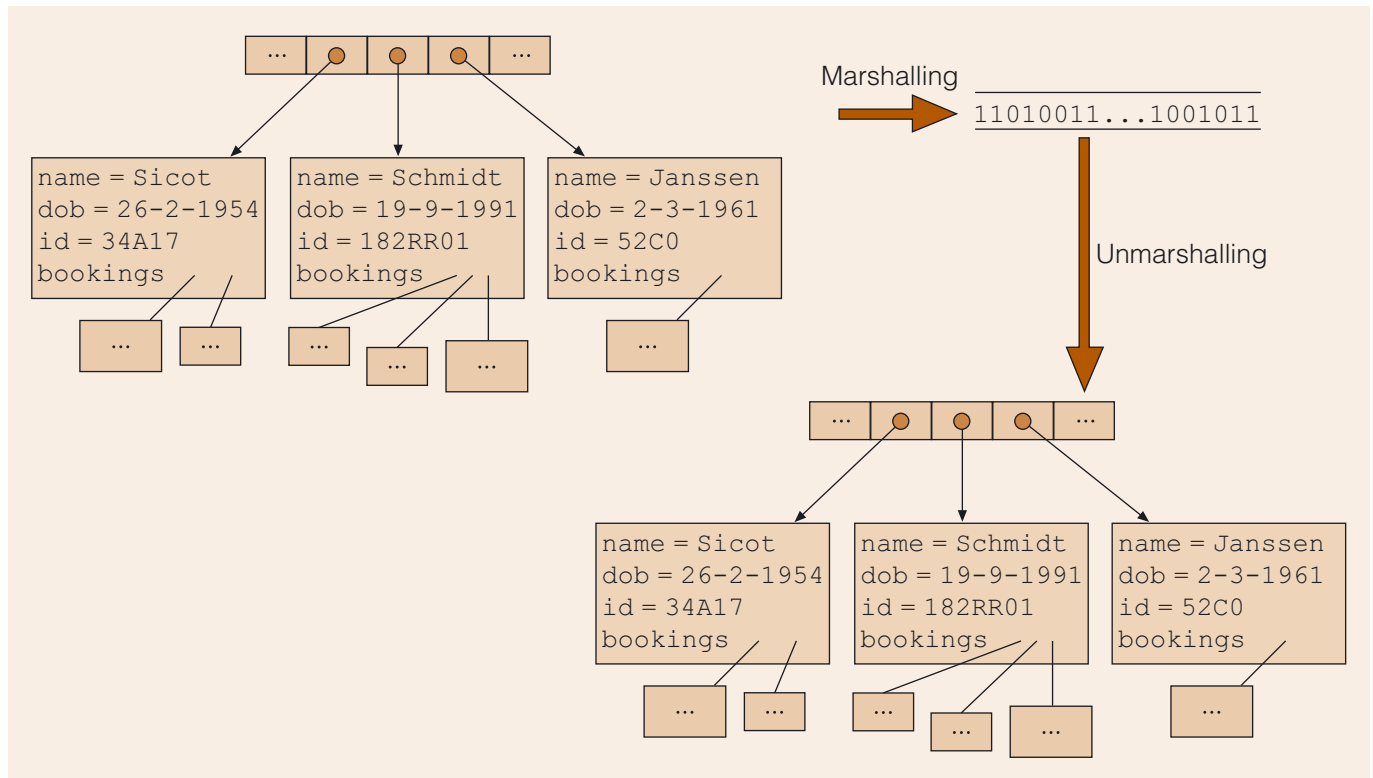


Figure 16 Data marshalling and unmarshalling

In Figure 16 we see the idea in action: the data structure in the top left corner is first flattened and the data values are converted to an external representation. This is then ready to be passed as a message. On arrival, the bit stream is converted again into the structure it held before, with all the values correctly formatted.

In Java this marshalling process has been given the name 'object serialisation'. We will return to Java's mechanism for this in *Unit 7*.

SAQ 4

- Which form of communication, i.e. synchronous or asynchronous, requires a buffer? Explain why.
- In an asynchronous message-passing system, the process that sends a message can continue processing. A receiving process is normally blocked until the *receive* method has completed. Explain how a non-blocking *receive* could be implemented.
- How can deadlock arise in communication? How can deadlock be resolved?

ANSWER.....

- In asynchronous message passing, a process sends a message with no regard as to whether the recipient is in a position to accept it or not. If it arrives before the recipient is ready to receive it, the message must be stored somewhere, i.e. buffered, until the recipient can read it.

- (b) By creating a separate thread that deals with the *receive*, the main thread can continue processing until it arrives at the point when it requires the data from the *receive* thread. The main thread and the *receive* thread would then be joined again.
 - (c) Deadlock can be the result of programming errors, or misunderstandings about the ordering of the *receive* and *send* operations with two or more processes waiting to receive from each other before they can continue. Most deadlock problems are overcome by using timeouts so that the call is abandoned if it appears unsuccessful after a specified time slot.
-

Client-server communication in Java

Message passing can be implemented in systems written in Java, with the facilities of the `java.net` package. In this section we first look at the way that two processes, a client and a server, can establish a connection, followed by a study of the `Socket` API in order to write some simple client-server systems.

6.1 Sockets

In order to communicate, client and server processes programmed in Java can use a **socket**. A socket is an abstraction of the network software that enables communication between two processes by providing various streams on which messages can be transferred. A socket is associated with a port and an IP address of a computer (as discussed in Subsection 5.3). In Figure 17 we show two sockets that form the connection between a client process and a server process.

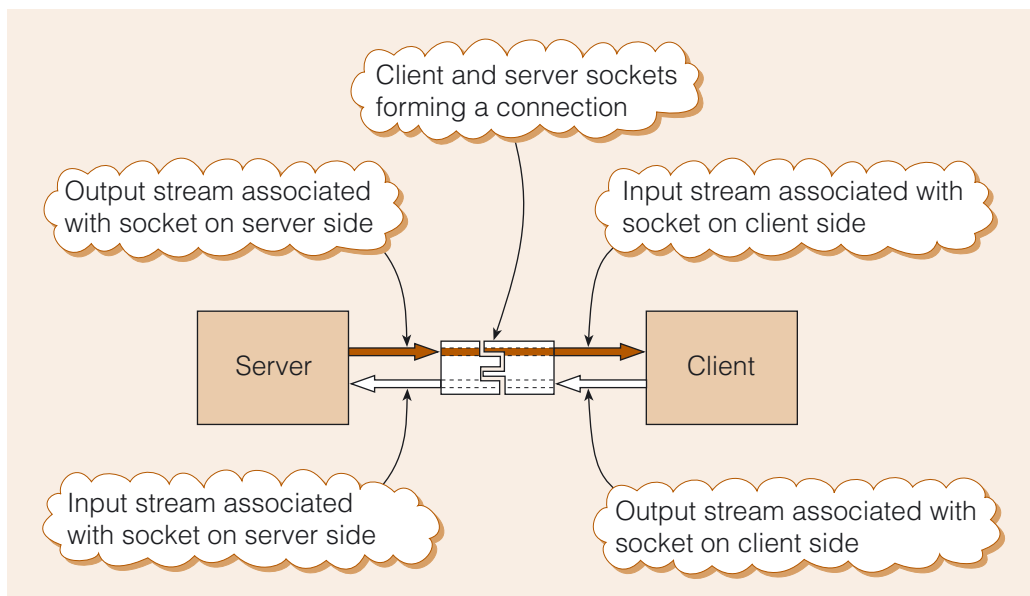


Figure 17 Two sockets with their input and output streams

Figure 17 shows how a client and a server are connected by 'plugging' together (joining) their respective sockets. It also shows that each socket has two streams: an input stream and an output stream. In Figure 17, the server is shown using its socket's output stream (shaded) to write a message. The client, in turn, uses its socket's input stream (shaded) to read the message.

In Figure 17 we have demonstrated the socket connection between a server and just *one* client. In reality a number of clients are able to communicate with a server at the same time, and so such a connection to the server would be set up for each client.

6.2 The `java.net` package

The `java.net` package is relatively large, but for the purposes of this course only a few of its classes will be examined: `InetAddress`, `ServerSocket` and `Socket`.

- ▶ `InetAddress`. An object of this class is used to represent an internet address. It has no public constructors; instead it uses `public static` methods that return an internet address when used anywhere in a program. One of these, `getLocalHost`, returns an `InetAddress` object representing the internet address of the local host machine (the machine on which the program is executing). Another, `getByName(String host)`, returns an `InetAddress` object representing the address of the host specified by name.
- ▶ `ServerSocket`. An object of this class is used by a server to 'listen' for connection requests from clients, and then establish a socket connection with each client. A `ServerSocket` object must be associated with a port where it can listen for a request using the method `accept`. The method `accept` has a dual function: it not only listens for a client request, but when this is received, returns a socket (of type `Socket`) so that the server now has the means to communicate with this client.
- ▶ `Socket`. An object of this class is used to establish a connection between client and server processes. Its constructors (it has more than one) associate a new socket with a specified host (IP address) and host port. The client and the server must each construct a socket, so the connection can be thought of as being made by these two sockets joining together.

A socket is also associated with both an input and an output stream to enable data to be communicated via the connection as explained for Figure 17.

In order to get hold of these streams we use the methods `getInputStream`, which returns an `InputStream` object, and `getOutputStream`, which returns an `OutputStream` object.

The methods `getInetAddress` and `getPort` return the internet address of the computer and the port of the socket respectively.

Note that the constructors for all three classes, as well as the methods `getLocalHost`, `getByName`, `accept`, `getInputStream` and `getOutputStream`, throw exceptions.

Sockets in a restaurant

In order to make the distinction between `ServerSocket` type objects and `Socket` type objects clear you may find the following metaphor helpful.

There are two types of waiter in a rather exclusive restaurant. One waiter (the `ServerSocket` waiter) stands at the door to welcome new diners. When new guests arrive, the `ServerSocket` waiter hands the guests over to a different waiter (a `Socket` waiter) who will take them to their table, and who will continue to look after them (and no one else) throughout their stay at the restaurant. While this goes on, the `ServerSocket` waiter remains at the door and continues to welcome new diners to the restaurant, allocating a `Socket` waiter to each new diner.

6.3 A simple 'date and time' server

In order to illustrate the working of a client-server program, we discuss the example of a simple system in which a client contacts a server program in order to obtain the current date and time.

Server program

The code for the class `DateServer` is as follows.

```
import java.util.Date;
import java.net.*;
import java.io.*;

public class DateServer
{
    static final public int portNumber = 4291;

    public DateServer() throws IOException
    {
        ServerSocket listener = new ServerSocket(portNumber);
        while (true)
        {
            System.out.println("Waiting for a client");
            Socket sock = listener.accept();

            // create an output stream through the socket
            OutputStreamWriter out =
                new OutputStreamWriter(sock.getOutputStream());

            String message = "Current date and time is "
                            + new Date() + "\n";

            // send message on the output stream
            out.write(message);
            out.close();
        }
    }
}
```

The code for class `DateServer` shows that, to start with, we need to create a `ServerSocket` object, which will be used to listen for clients. Having done that, a `while` loop is started, which will execute continuously. In this `while` loop the following statement is important:

```
Socket sock = listener.accept();
```

The `accept` method is a blocking method, which listens for clients that wish to make a connection with this server program. As soon as a client is detected, a `Socket` object is created, which then makes the connection between the server and this client.

The output stream is obtained from the socket, and this is used to create a writing stream, the `OutputStreamWriter` object `out`. A suitable message is created using the `Date` utility from the utilities library, and this is sent to the client with the statement

```
out.write(message);
```

Having completed this task, the stream is destroyed, and the server continues by listening for other potential clients.

It is easy to confuse the various terms associated with setting up the communication. In particular, a `ServerSocket` object may be confused with the server's `Socket` object. The former does not have input and output streams: its role is to provide the facilities to listen for a client's request to communicate and then to create the server's `Socket` object. The `Socket` object on the server is connected to the client's `Socket` object, and the communication takes place through this pair of sockets: one on the server side and one on the client side, as was shown in Figure 17.

Client program

The code for the client is as follows.

```
import java.io.*;
import java.net.*;

public class DateClient
{
    static final public int portNumber = 4291;

    public DateClient() throws IOException
    {
        Socket sock = new Socket("**SomeRealIPAddress**", portNumber);

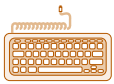
        Reader isread = new InputStreamReader(sock.getInputStream());
        BufferedReader input = new BufferedReader(isread);

        String message = input.readLine();
        System.out.println("Message received is: " + message);
    }
}
```

The class `DateClient` has a constructor but no other methods. The constructor creates the `Socket` object on the specified IP address for the server, on the port that is being used by the server to listen for the client.

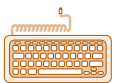
Note that the IP address would be replaced by the correct address for the server, be it in numeric or symbolic form, as discussed in Subsection 5.3. During testing it is often useful to run both the client and the server on the same machine, and in that case we could use the method `InetAddress.getLocalHost` in order to obtain the address of the host machine that we are working on.

Having created a `Socket` object, the input stream object is obtained in order to be able to read the data that is being sent to the client. This input stream object is wrapped into a `BufferedReader` which is more convenient to read from. The method `readLine` is then invoked to get the data from this stream, which is then output to the standard output device.



Activity 5.2

Running the date server.



Activity 5.3

Weather forecast.

SAQ 5

- Describe the difference between a `ServerSocket` object and a `Socket` object.
- Describe how the connection between a client and a server is made. How do client and server communicate once this connection is set up?
- Explain whether the `DateServer` application is synchronous or asynchronous.

ANSWER.....

- (a) A `ServerSocket` object listens at a particular port for requests from a client.

A `Socket` object forms part of the connection by means of which client and server communicate. Both the client and the server have a socket which, together, form the connection. Each socket provides input and output streams that enable the client and the server to receive data from and send data to the connection.

- (b) The client creates a socket based on the internet address of the server's computer and the port on the server's computer at which the connection will be made. This act of creating the socket can be thought of as requesting a connection with the server.

The server creates a `ServerSocket` object associated with the port on its own machine at which the connection with the client is to be made. The method `accept` is invoked on this server socket, which causes the server to wait for the client's request. When this arrives, `accept` makes the connection with the client and returns the server's connection socket object through which it will communicate with the client.

The client and the server use the output and input streams associated with their own socket to the connection for sending data to and from each other.

- (c) It is asynchronous because the sending process can send its message, irrespective of whether the receiving process is ready for it. An example of this is the statement

```
out.write(message);
```

executed by the `DateServer`, which is equivalent to a `send`. Similarly, the receiving process can pick up the message as and when it requires the message (which may have been buffered).

6.4 A chat room server

The `DateServer` example introduced the basic principles of client-server programming in Java. However, any real form of communication is likely to require more sophisticated ways of working. For example, the `DateServer` can be in communication with only one client at a time, whereas in most real systems the server should be able to handle communication with a large number of clients concurrently. Also, in the date example, we saw the client only receiving data from the server, but there was no sense of an ongoing dialogue between the two processes.

We will now look at a more interesting and complex example, based on the example by Bishop (1998, pp. 417–22). It involves a server for a chat room, which supports a number of users who are chatting. This application has the following requirements.

- ▶ The server must be capable of handling a number of clients.
- ▶ When a user joins or leaves the chat room, their entrance/exit should be announced to all the other chat room users.
- ▶ Any chat room user is allowed to join in with the conversation, and their contribution should be forwarded to all the other users.

Figure 18 shows an example of what such a chat room dialogue would look like from a GUI client's view.

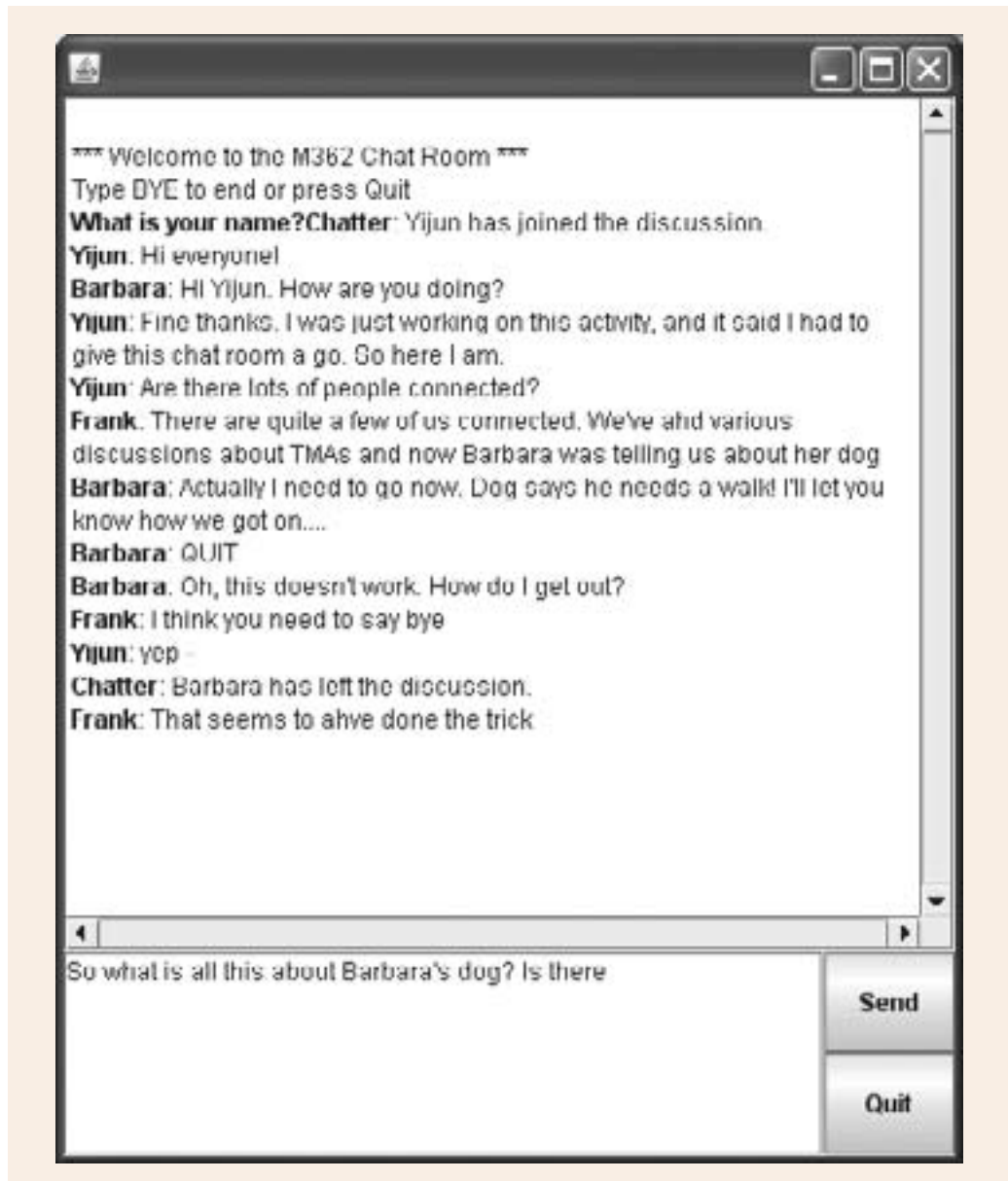


Figure 18 Screenshot of a chat client

The client's view shows the dialogue between a number of clients, and the attempt of a client to leave the chat room. Client input appears in the pane at the bottom of the screen until the user sends it. The dialogue between users of the chat room appears in the top pane, where 'Chatter' represents the server, which 'manages' the discussion. Currently Yijun is writing a new message relating to Barbara's dog, and this message has not been sent to the chat room yet.

Server program

The code for the class `ChatServer` is as follows.

```
public class ChatServer
{
    private static LinkedList<Socket> clientList =
        new LinkedList<Socket>();

    public ChatServer() throws IOException
    {
        int port = 8190;
        ServerSocket listener = new ServerSocket(port);

        while (true)
        {
            Socket client = listener.accept();
            ChatHandler handler= new ChatHandler(client);
            handler.start();

            clientList.add(client);
        }
    }

    static synchronized void forward(String message, String name)
        throws IOException
    {
        // sends the message to every client including the sender
        Socket s;
        PrintWriter p;

        for (int i = 0; i < clientList.size(); i++)
        {
            s = (Socket) clientList.get(i);
            p = new PrintWriter(s.getOutputStream(), true);
            p.println(name + ": " + message);
        }
    }

    static synchronized void remove(Socket s)
    {
        clientList.remove(s);
    }
}
```

The class `ChatServer` has a list of clients, `clientList`, a constructor and two methods, `forward` and `remove`. The first issue to resolve for this application is how to make sure that the server can keep track of all the clients, since its main task will be to relay messages to all the chat room users. This is resolved by using the linked list, `clientList`, in which all the sockets to communicate with clients are stored.

The constructor begins by setting up a `ServerSocket` on the port. This `ServerSocket` is used in the `while` loop to listen for clients, using the `accept` method, as we saw before. Each time a client has been detected, a socket is created to handle the communication with this client. Once this is done, the client is handed over to a separate handler thread, which will then handle the actual communication with this client. Finally, the client is added to the list of clients.

It is interesting to note that the design of this constructor shows that the two tasks – listening for clients, and handling a client – have been separated, split between two classes, `ChatServer` and `ChatHandler`. This is an example of modularisation, and each class carries out one specific task, rather than a number of different tasks. This increased modularity is necessary in order to be able to look after a number of clients concurrently.

The `forward` method has a `for` loop to process all the clients, and sends a given message to each of the clients. Note that both `forward` and `remove` have two modifiers: `static` and `synchronized`. The methods access the `clientList` data field which has been declared as `static` in order to ensure that there is one, and only one, list of clients, and not several local copies of the list (which may all be slightly out of date). To lock access to a `static` data field the method must be both `synchronized` and `static`, so that the lock owned by the *class* is used.

See also *Unit 4*, Activity 4.5 for a discussion of the lock held by a Java class.

The code for class `ChatHandler` is shown below.

```
class ChatHandler extends Thread
{
    /* the ChatHandler class is called from the chat server:
    * one thread for each client coming in to chat
    */
    private BufferedReader in;
    private PrintWriter out;
    private Socket toClient;
    private String name;

    ChatHandler(Socket s)
    {
        toClient = s;
    }

    public void run()
    {
        try
        {
            openStreams();
            // welcome new client to the Chat Room
            out.println("*** Welcome to the M362 Chat Room ***");
            out.flush();
            out.println("Type BYE to end or press Quit");
            out.flush();
            out.print("What is your name?");
            out.flush();

            String name = in.readLine();
            ChatServer.forward(name + " has joined the discussion.",
                               "Chatter");
```

```

        // read lines and send them off for forwarding

        while (true)
        {
            String s = in.readLine();
            if (s.startsWith("BYE"))
            {
                ChatServer.forward(name + " has left the discussion.",
                                   "Chatter");

                break;
            }
            ChatServer.forward(s, name);
        }

        ChatServer.remove(toClient);
        toClient.close();
    }
    catch (Exception e)
    {
        System.out.println("Chatter error: " + e);
    }
}

private void openStreams() throws IOException
{
    in =
        new BufferedReader(new InputStreamReader(toClient.getInputStream()));
    out = new PrintWriter(toClient.getOutputStream(), true);
}
}

```

The most important observation to make about the `ChatHandler` class is that it extends `Thread`. This means that when a new `ChatHandler` object is created, this effectively creates a new thread. The thread can then be put in charge of conducting the communication between this particular client and the chat server, as is outlined in the `run` method.

The `run` method starts by opening the appropriate streams for communication, followed by displaying various welcome messages to the client. It reads input from the client, the client's name, and makes an announcement to the chat room, using the `forward` method. It then continues with a `while` loop, which, unless the client sends the `BYE` message, continually picks up messages from this client and requests that the chat room object forwards these messages.

Note that, in order to invoke a `static` method, we need to include the class name, as in

```
ChatServer.forward();
```

Client program

We now look at the client side of the application. We will limit our explanation to a client version that is simpler than the GUI version that was shown in Figure 18. However, the basic functionality remains the same.

The client side also has to be threaded, in order to be able to deal with the two tasks for the client: displaying output from all the other clients, and sending this client's input to the chat server.

```

import java.net.*;
import java.io.*;

public class ChatClient
{
    Socket socket;
    static int portNumber = 8190;
    BufferedReader fromServer;
    PrintWriter toServer;

    // creates a new instance of ChatClient
    public ChatClient()
    {
        try
        {
            joinChatRoom();
            Thread input = new InputThread(toServer);
            input.start();
            printMessages();
        }
        catch(Exception e)
        {
            System.out.println("Problems communicating with the chat room");
        }
    }

    public void joinChatRoom() throws IOException
    {
        socket = new Socket(InetAddress.getLocalHost(), portNumber);
        fromServer =
            new BufferedReader(new InputStreamReader(socket.getInputStream()));
        toServer = new PrintWriter(socket.getOutputStream(), true);
    }

    public void printMessages() throws IOException
    {
        String message = "";
        while (message!=null)
        {
            System.out.println(message);
            message = fromServer.readLine();
        }
    }
}

```

The class `ChatClient` has a constructor and two methods: `joinChatRoom` and `printMessages`. The content of the `joinChatRoom` method is familiar to us – all it does is to create the socket on the client's side, and sets up the streams for the communication. The method `printMessages` operates by continually reading from the stream coming from the chat server, and printing out the forwarded messages. So this would print out all the messages, as we saw in the top section of Figure 18.

Looking at the constructor, we see that after invoking the `joinChatRoom` method, a new thread is created to handle the input from this particular human client. The code for the `InputThread` class is shown below.

```
import java.io.*;

public class InputThread extends Thread
{
    PrintWriter toServer;

    // creates a new instance of InputThread
    public InputThread(PrintWriter to)
    {
        toServer = to;
    }

    public void run()
    {
        Reader isread = new InputStreamReader(System.in);
        BufferedReader input = new BufferedReader(isread);
        try
        {
            String message = input.readLine();
            do
            {
                toServer.println(message);
                message = input.readLine();
            }
            while (message != "BYE");
        }
        catch(IOException ex)
        {
            System.out.println("trouble contacting the chat room");
        }
    }
}
```

The class `InputThread` inherits from `Thread`, and has a `run` method. Since we are dealing with the non-GUI version here, this thread is picking up the input from the standard input stream, `System.in`. The main functionality of this class is again in a `while` loop, this time a `do-while`. It picks up a string message that is typed in on the standard input stream, and sends this to the server. The server will then handle the forwarding of this message.

Note that in the case of this code running on behalf of a human user who never contributes to the discussion but only observes, a so-called 'lurker', the execution remains blocked on the `readLine` statement.



Activity 5.4

Running the chat server.



Activity 5.5

The GUI chat client.

SAQ 6

- What are the advantages of having a multithreaded server, rather than a single-threaded server?
- In the case of implementing a client for a chat server, why does it make sense to implement this with two threads?

ANSWER.....

- (a) If a server can handle only one client at a time, the server would be rather unsatisfactory for most applications. Each client trying to communicate with this server would have to wait until the previous client has finished. This would be very unresponsive.
- (b) In the case of a chat client, there are two quite separate actions going on:
- 1 forwarded messages from fellow chat clients (as sent by the chat server) are being received and printed to the screen;
 - 2 contributions from the chat client itself are being sent to the server so that they can be forwarded.

The two separate actions are going on continually, and it cannot be determined beforehand at exactly which point the client would want to make a contribution, nor when all the fellow chatters want to make a contribution.

Exercise 2

Using pseudocode, give an outline design of the code for a server which can communicate with several clients, rather than just the one.

Discussion

The design for a multithreaded server would look as follows.

```
listen for clients on ServerSocket
while (true)
{
    create a socket for newly arrived client
    create a thread to handle communication with this client
    start the thread
}
```

Paradigms for distributed communication

In Sections 5 and 6 we studied some basic concepts for communication in distributed systems, and we looked at the way we can use sockets to implement message passing in Java.

In the current section you will see two different distributed communication paradigms: the remote procedure call paradigm and the distributed object paradigm.

A **paradigm** can be thought of as a *pattern*, a *model* or an *example* – it is also another form of abstraction. Hence we want to emphasise that we will concentrate on the underlying ideas of these different forms of communication, rather than their details (which we will cover in later units). Looking at models at a high level is particularly useful in the area that we are studying here, where new acronyms appear regularly and where it is quite easy to feel somewhat lost in the myriad of potential technologies. By studying the paradigms, we can highlight the main issues, and you will therefore be in a better position to understand the relevance of new mechanisms should they come your way.

7.1 Remote procedure call (RPC)

The **remote procedure call (RPC)** model draws our attention to an important dimension of software: whether or not the software developer needs to be aware of the fact that a procedure (or method) is executing on a remote computer. We therefore say that one of the main goals of RPC is to make communication **distribution transparent**, in that it aims to hide the complexities of a distributed system.

Procedure calls

To explain this, we first must understand what is meant by a **procedure call**. A 'normal' procedure call is shown in Figure 19, where a procedure (here the procedure `doThis`) is called at a certain point in the program.

A 'procedure call' is the same as a 'method invocation'. The terminology stems from the era (before object-oriented programming languages) when programming languages (for example C, Pascal or Modula 2) were predominantly procedural.

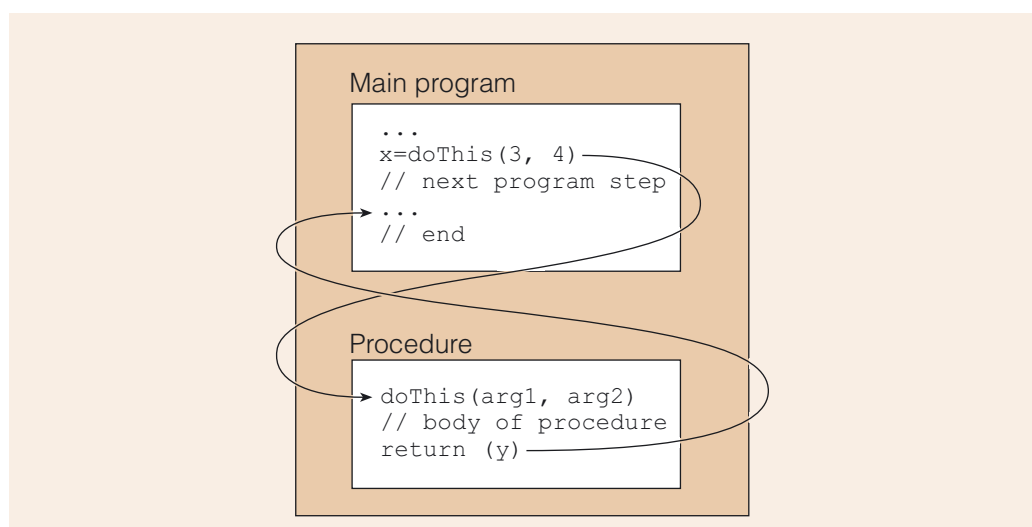


Figure 19 A procedure call

To keep things simple we have ignored the possibility of communication or other problems occurring.

When the procedure is called, the code associated with the procedure is executed, and once this is finished and any necessary data has been returned, the execution of the main program continues with the next statement.

Now imagine that the procedure `doThis` cannot be executed on the local machine by the current process, but has to be executed by a process at a remote location. Instead of the straightforward call to `doThis` we would require the sequence of steps below.

- ▶ Set up a two-way communication, which involves connecting sockets and creating input and output streams.
- ▶ Package a message which specifies the procedure call (that is, gives the name of the procedure and any arguments that it takes). Note that this packaging requires a protocol – there must be a format, which has to be known in advance, so that the processes at both ends know how to pack and unpack this message.
- ▶ Send the message containing the request to the remote process. The calling process now waits.
- ▶ On receiving the request, the remote process does the following:
 - unpacks the message, again using the protocol;
 - executes the required procedure;
 - packages any return value, using a second part of the protocol;
 - sends the result back to the calling process.
- ▶ Finally, the calling process receives the result and unpacks it, and the call is complete.

Any application can be written to work like this if we wish, but there are considerable disadvantages. The programmer has to write code to do all the packing and unpacking, as well as setting up the communication channel. This involves a lot of relatively low-level coding which is time consuming and error-prone. Moreover it is specific to this particular application and therefore unlikely to be reusable. So doing things this way is a poor use of resources, as well as increasing the likelihood of program defects.

What is required is a way to make the procedure call using a *general* scheme, not specific to any particular application, and to have the details taken care of automatically so that the programmer no longer has to worry about them. The programmer should be able to think at a *higher level of abstraction*: not ‘How will I set up the sockets and streams, what protocol will I use, and what code must I write to pack and unpack the request and response?’, but simply ‘What procedure call do I want to make and to what remote location?’ This is the goal of RPC.

Remote procedure calls

In the RPC model the idea of procedure calling is extended so that it is possible to call a procedure even if the process that will execute that procedure is remote. This model is shown in Figure 20.

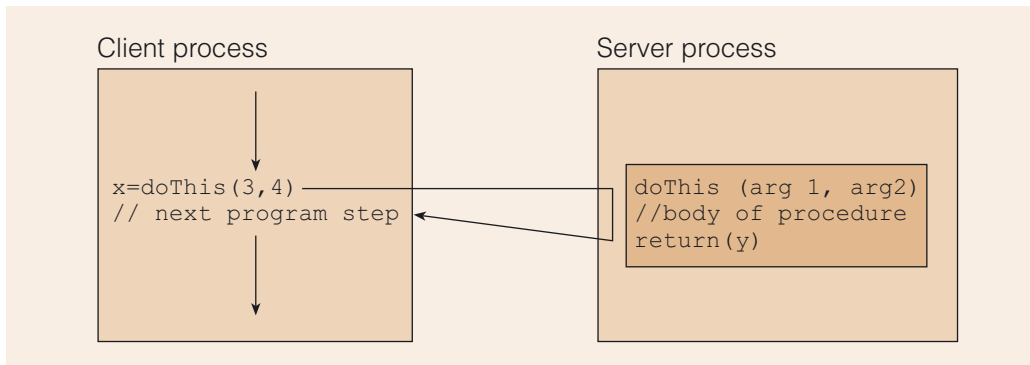


Figure 20 A remote procedure call

Figure 20 shows the same example as Figure 19 of calling the procedure `doThis`, but now in a distributed setting, where the client calling the procedure is on a different machine from the server where the procedure is carried out.

The main advantage of RPC is that it is easier for the developer to use than the low-level programming with sockets and streams because it offers a higher level of abstraction, allowing the developer to focus on the bigger picture. Hence it is less prone to errors, easier to debug and, over time, easier to maintain.

If we compare the main idea of the basic message-passing model we can say that message passing is essentially a **data-oriented** form of communication: the focus is on the data that is being passed around, and actions will be triggered as bits of data are exchanged. In contrast, the RPC model is essentially **action-oriented**: the action of calling procedures is the focus, and data is passed around in the form of parameters to these actions.

The RPC service layer

At the level of the application, an RPC will look like the example discussed above – no different from an ordinary procedure call. However, when RPC is actually used in practice, the RPC will be converted to a message exchange with a server. Effectively, RPC works by adding a layer, the RPC service layer, to the system. This is shown in Figure 21 overleaf. (The roles of points A to E are discussed in the next subsection, on implementation.)

In Subsection 3.2 we discussed the layered approach for systems, and we briefly looked at the OSI model for network communication. Figure 21 is a condensed form. The lower-level protocols cover the layers typically being taken care of by the operating system: at transport level and below. The RPC service layer covers the session layer and presentation layer of the OSI model.

What this means is that at the top layer, which is the level where programmers usually develop their applications, an RPC call can be made. This top layer will then pass on the RPC call to the layer below it, here the RPC service layer. All the necessary work (see the section below on implementation) will be done in this layer in order to make RPC work. The level below that will then actually send the message across to the server. So the same sockets and streams are still at work, but all these details are shielded from the application layer.

RPC is one of the first examples of **middleware**. In general, the term middleware is used to indicate a layer of software that aims to mask the heterogeneity of the underlying platforms in a distributed system such as networks, hardware and operating systems. By building a layer of software above the heterogeneous systems, a uniform platform is created on which the distributed applications can run. This then simplifies the development of distributed applications because it is not necessary to port each application to a range of operating systems and platforms.

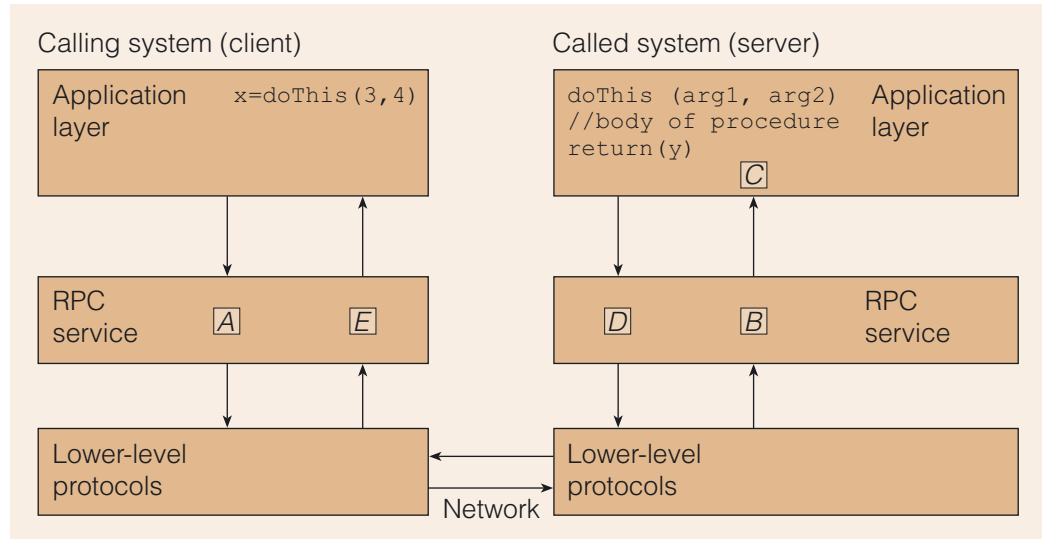


Figure 21 The RPC service layer

Implementation of RPC

An important issue in the implementation of an RPC service is how to deal with the potential problems caused by failures of computers, or failure of the network.

In Subsection 5.2 we discussed the use of timeouts as a means for processes to recover from such problems. In order to implement timeouts the RPC service adds two timers and it creates an ID for the RPC call. Here is how an RPC call uses the various timers when making a call.

Points *A* to *E* are as indicated in Figure 21.

- ▶ When the calling system creates an RPC call the RPC service creates a unique ID for this call, marshals the arguments into some suitable format, and sets the timer at point *A*.
- ▶ The data is passed as a message through the lower levels.
- ▶ At point *B*, the arguments are unmarshalled, and a note is made of the ID for this RPC call.
- ▶ The required method is called and executed at point *C*. The return from this procedure call is at point *D*.
- ▶ At point *D*, the return arguments are marshalled again, ready for transport, and a timer at point *D* is set.
- ▶ At point *E*, the return arguments are unmarshalled, the timer at point *A* is disabled, and an acknowledgement is sent to point *D* that the RPC with the specified ID has arrived safely.
- ▶ The timer at point *D* is disabled.

The timers are there to make the service more robust, in the case of disruption to the network, or failure of either the client or the server. For example, if there is congestion on the network, the timers at points *A* or *D* will expire. The RPC service will retry the call from point *A* several times. If there is a problem with the network, it may be unclear whether or not the original call actually got through or not. This is where the RPC ID comes in useful, because, by including it with any repeated call, the called system can detect whether or not it is dealing with a repeat request. If the original call had been successful, then processing at point *C* would already have taken place, including the sending back of the argument from point *D*. If the sending back from point *D* turns out to be unsuccessful this can be detected by the called system by inspection of the RPC ID. If the called system detects a repeated RPC call, it does not have to do the processing at point *C* all over again, but it can go straight to point *D* and resend the message.

Failure of the client can happen at any point, even just after it has initiated an RPC call. The RPC call will simply go ahead, now termed an **orphan**. Various operations may take place at the server end, potentially changing data on request of the failed client process. The failure of the client is noticed when no acknowledgement ever arrives at point *D*. It may be possible to undo certain state changes at the server side if the requesting client has failed.

Failure of the server can be problematical. The server may potentially fail at point *B*, point *C* or point *D*, but this will always be noticed due to the timer at point *A* expiring. When the failure is noticed, the client may decide to repeat the call (using the same ID). At this point, however, it will be unclear how much progress the original procedure call had made and whether the operation at *C* had actually taken place or not. Since it is unlikely that the server permanently stored the ID of the RPC call just before failure, when the server is restarted it will not be possible to check whether the operation had taken place. Hence this scenario of the client repeating the call in the case of server failure may lead to changes to the data being made twice. It depends on the nature of the application how important it is that this should be avoided.

RPC in use

The initial idea for RPC was first introduced in 1976, and its first implementation started to be used during the 1980s. RPC is now widely used in network applications. There are a number of variants on the basic idea of RPC, notably RMI which we discuss in the next subsection and in more detail in *Unit 7*.

7.2 Distributed object paradigm

The RPC mechanism just described is a step up from the basic message-passing paradigm – that is, it operates at a higher level of abstraction and some of the implementation details are hidden from the application-level programming environment. Here we discuss another example of such a higher-level paradigm, called the **distributed object paradigm**.

When working in an object-oriented environment, work gets done through objects invoking methods on other objects. However, there is no reason why object orientation should be limited to local applications, running on the same host. When dealing with distributed systems, we can extend the idea behind object orientation by including distributed objects that can be dealt with in much the same way as local objects.

We say that **local objects** are objects whose methods can be invoked only by a **local process** – a process that runs on the machine on which the object resides. On the other hand, a distributed object is one whose methods can be invoked by a **remote process** – a process running on a different machine, which is connected to the one on which the object is located. Objects provide methods, and through the invocation of these methods, applications can gain access to the services of the distributed objects.

Remote Method Invocation

Remote Method Invocation (RMI) is the object-oriented equivalent of RPC. With RMI, a process can invoke methods on an object, as it would when dealing with objects on the local machine. In that respect, RMI is an example of both RPC and distributed objects.

Figure 22 shows an example of objects invoking methods on remote objects (say, object A on object X) in much the same as they would if the objects were local (say, object A on object D).

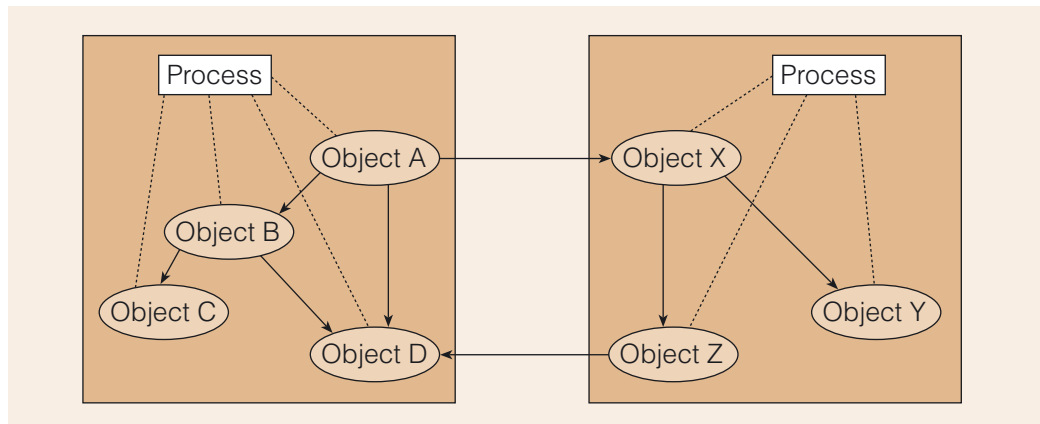


Figure 22 Remote Method Invocation, showing local and distributed objects

As with RPC, arguments may be passed with the invocation of methods, and when a call completes, a value may be returned.

RMI is essentially a Java mechanism, and it assumes that both parties (the invoking object and the object with its method being invoked) are implemented in Java. Therefore, RMI is a **homogeneous** form of middleware, due to the homogeneity of the programming language that is used during the communication.

In the Java EE framework we use the Enterprise JavaBeans (EJB) component model, which is a model for distributed objects. EJBs have been designed to interact with other components in a standard way, allowing us to ignore the difference between enterprise components located on the same machine or located on remote machines. Java EE also uses RMI as a form of communication between the distributed objects

In *Unit 7*, where we study the business tier, we will study the ideas of EJB and RMI in more detail.

SAQ 7

- Is RPC an asynchronous or synchronous form of communication? And RMI?
- What is the role of the timers in the RPC implementation?
- Distinguish between data-oriented, task-oriented and object-oriented forms of communication. Give examples of each.

ANSWER.....

- RPC and RMI are both synchronous forms of communication, because the calling process is blocked while the call is being made, and it can continue only when the call returns.

- (b) The main role of the timers is to ensure that processes do not 'hang' indefinitely if there are problems in the communication. A timer expiring is essentially a timeout, and is a signal to the RPC service that some form of action needs to be undertaken to remedy the situation.
- (c) Message passing is data-oriented: the focus is on the data that is exchanged between the processes, and it is only as a result of data being exchanged that things get done.

RPC and RMI are action-oriented: the focus is on the work that needs to be done, which can be expressed through calling procedures or invoking methods.

RMI is also object-oriented: local objects and remote objects are treated in the same way, and methods can be invoked on either.

Exercise 3

You have been hired to work as a consultant on an application for online auctioning. At any given time there are thousands of auctions in progress. Users can request to see a list of items currently up for auction, and place bids. Each auction will run for a number of days, and whoever has bid the highest wins the item, and has to pay his/her bid. Users can opt to be informed of bids placed on items that are being auctioned, even if they have not placed a bid.

- (a) Explain what might be the benefits of using RPC for such an application.
- (b) Assuming that you are using RPC, give examples of procedures that the auction system could make available for remote calling.
- (c) Now imagine that you decide to use RMI for the auction system. Give examples of the classes you would need in order to develop the system so that instances of these objects could be either local or remote objects. For each class give the signature for a suggested method.

Discussion

- (a) This will be a complex system, with many different issues having to be communicated. If using simple sockets, there would be many different types of message, and it would be a considerable effort to marshal and unmarshal these messages. It would be much easier if you could use RPC, as these issues could be resolved by the RPC service layer, and you, as the developer, would be able to concentrate on when to call the appropriate procedures.

Furthermore, RPC uses protocols (based on the timers and IDs as discussed) which enable a reliable form of communication, such that we can be confident that a message has arrived. This is important in an auction system, where the bidder wants to know for certain that her/his bid has gone through.

- (b) Here is our suggestion for the procedures that the auctioneer's system should make available for remote calling.

```
procedure display_items_in_auction
procedure place_bid(item)
procedure register_interest_in_item(item)
procedure pay_for_item(item)
```

- (c) Here is our suggestion for a number of classes that would be modelled in the auction system.

```
AuctionSystem
    registerUser( )
    requestListOfItems( )

AuctionItem
    registerInterestInItem( )
    placeBid( amount )

Bid
    setAmount( )
    confirm( )
    pay( )

Payment
    selectPaymentMethod( )
```

There are many equally valid alternatives that you may have suggested instead.

8

Advanced forms of communication

In this section we consider some forms of communication for distributed systems which operate at a higher level of abstraction than RPC and RMI. Figure 23 illustrates the idea of the communication models we have studied so far, and those still to come, in terms of their increasing level of abstraction.

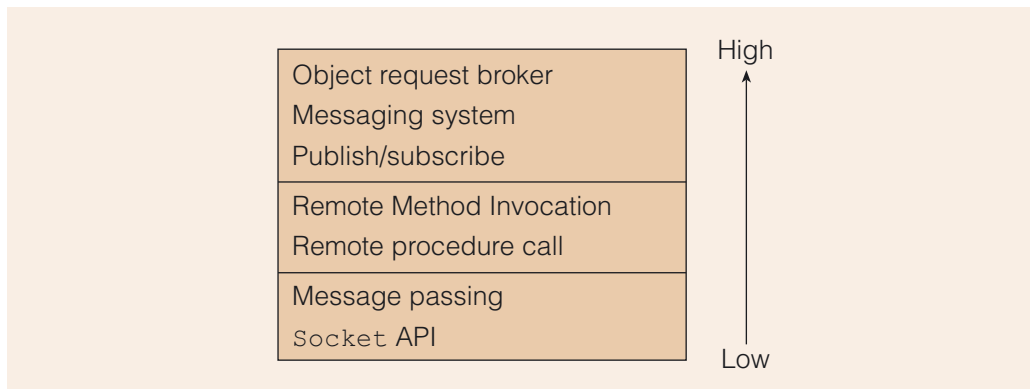


Figure 23 Abstraction levels for models of communication

In this section you will be introduced to the object request broker, messaging system and publish/subscribe models of communication. Our aim at this stage is to give you a flavour of these different paradigms so the discussions are brief. Where appropriate we will indicate in which unit you will see that particular type of communication in more detail.

We will also look at a form of communication that can be used in P2P systems, which is slightly outside the discussion in terms of abstraction levels.

This section will conclude with a comparison of the different forms of communication studied in the unit. In particular we will discuss the issues to be considered when choosing a type of communication, the circumstances under which we might choose one type rather than another, and the trade-offs to be made.

8.1 Object request brokers

The **object request broker (ORB)** model is another example of using distributed objects. It differs from RMI in that a process makes a request to the ORB for the services of a remote object rather than by directly addressing this remote object as in RMI. The ORB directs this request to the appropriate object, as illustrated in Figure 24.

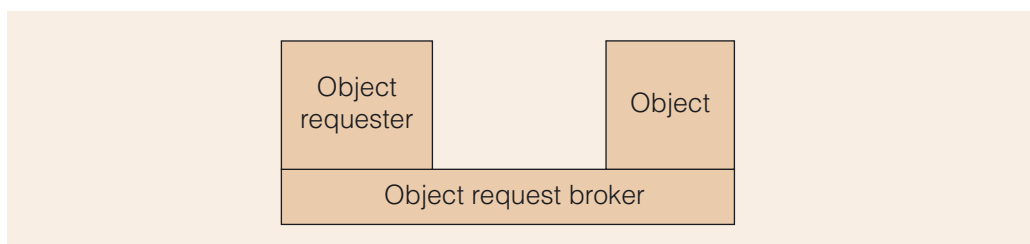


Figure 24 The object request broker

The ORB is in effect an additional layer of software and functions as middleware, allowing an application, as an object requester, to access one or more (local or remote) objects.

The ORB may also act as a mediator for heterogeneous objects, making it possible for objects to communicate even if they don't '*speak the same language*'. That is, the objects may be implemented in different languages (C++ and Java, for example) and running on different platforms, making this paradigm particularly suitable for dealing with **heterogeneous** systems.

In *Unit 11* we will study CORBA, the common object request broker architecture, as an architecture for heterogeneous systems.

8.2 The messaging system paradigm

The **messaging system paradigm** is a more sophisticated version of the basic message-passing paradigm studied in Sections 5 and 6. This paradigm is also referred to as the message queue system, or message-oriented middleware (MOM).

In the messaging system paradigm the messages do not go straight from the sender to the receiver, but they go via an intermediary object. This is shown in Figure 25.

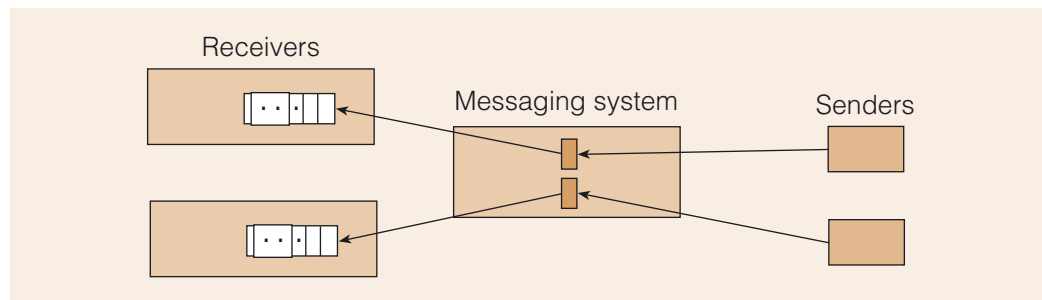


Figure 25 A messaging system

A sender deposits a message with the *messaging system*, which acts as the intermediary, and the intermediary then looks after the message from that point onwards. This form of communication is therefore asynchronous – the sender is not blocked upon sending a message, but can continue with other processing. The processes are said to be loosely coupled.

The job of delivering the message to the appropriate receiver is taken on by the messaging system intermediary, by forwarding the message to the queue associated with the appropriate receiver. If the receiver process is not up and running, or if it has no space in its buffer to receive another message, the messaging system will hold on to the message until the receiver is ready to receive it. So this is a reliable form of communication.

8.3 Publish/subscribe messaging model

The **publish/subscribe messaging model** is a variant of the messaging system model. In this model, each message is associated with a specific topic or event. This model is also sometimes referred to as the **event/notification model**.

An object that generates certain *events* will *publish* the type of events that it is capable of generating, and other processes are allowed to observe. Applications interested in the occurrence of a specific event may *subscribe* to messages for

that event. When the awaited event occurs, the process publishes a message announcing the event or topic. This is called the *notification*. The middleware messaging system distributes the message to all subscribers for that event.

The publish/subscribe messaging model is a very useful form of group communication. Any number of processes are able to register their interest in an event, and the *publish* operation allows a process to **multicast** to a group of processes. To multicast means that the sender can send information to a group of destinations simultaneously, in contrast to a **unicast** in which the sender and receiver are in a one-to-one relationship, and the sender sends a message to each individual recipient.

The publish/subscribe messaging model is a form of communication that would suit an auction scenario, such as that contemplated by the Music Store Internet Café, particularly well. Here is how this model could be used for an auction session in which just one particular item is being auctioned.

- ▶ The auctioneer process indicates that an auction for an item has started by sending a 'begin-auction' event message.
- ▶ The auctioneer process can publish a 'news-item' event message, to announce progress on the bidding for an item.
- ▶ A participant subscribes to an 'end-auction' event message.
- ▶ The auctioneer process subscribes to 'new-bid' event messages.
- ▶ If a participant places a bid, this will result in a 'new-bid' event message, which gets forwarded to the auctioneer process.
- ▶ A participant can register their interest in an item by subscribing to 'news-item' events.
- ▶ On conclusion of the auction, an event announcing the end will be published by the auctioneer, which is sent to all the participants.

An example of this form of communication is the Java Message Service (JMS). We will return to this topic in *Unit 9*.

8.4 Peer-to-peer file sharing

The communication model for peer-to-peer systems is altogether different from those we have discussed so far. Since there is no central server, and because each peer is both a client and a server, the communication model is essentially cooperative. Furthermore, peers may appear and disappear at any given time, so the communication model must be able to deal with this very ad hoc network configuration.

Peer-to-peer systems have been mainly geared towards the sharing of large data files. You already saw the example of the Napster server, primarily geared towards sharing MP3 files between peers. Video files, much bigger than MP3 files, are also targeted for sharing between peers.

An interesting style of peer-to-peer file sharing was developed with the **BitTorrent protocol**. Until this protocol emerged, peer-to-peer systems were rather sluggish when it came to large data files, because it can take ages for a peer to send a file to another peer. The designers of BitTorrent decided that large files could be split into smaller chunks so that each peer, once it has downloaded a chunk, can send this chunk on to others. Because there are now a number of peers, each uploading a (different) chunk of the original file, there is a great improvement in the performance. This is illustrated in Figure 26.



BitTorrent animation.

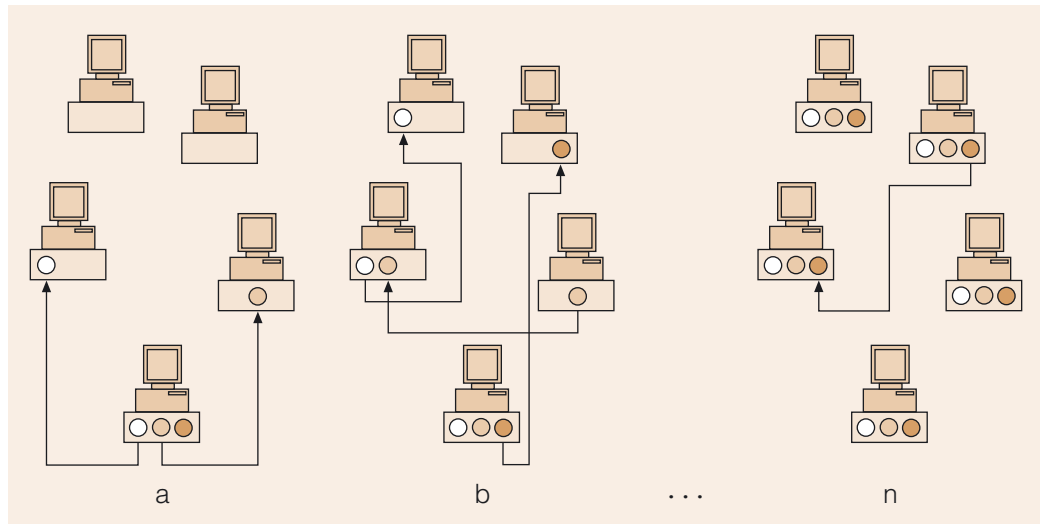


Figure 26 A 'swarm' at work to distribute a file

In Figure 26 in step (a) we see the initial scenario where one computer (at the bottom) has the complete file 'bittorrented' into three parts. This computer starts by sending two of the parts to neighbouring peers. In step (b), this process continues, where the third part is sent to a different computer, while some of the other peers are each sending the part they obtained to other peers. In step (n) the process completes when the last peer receives the final part. The host computer transmitted each part only once, thus sending the file as a whole only once.

In the BitTorrent protocol the term **swarm** is used to denote a group of connected computers that have either a portion of the file or the file in its entirety and are in the process of sending or receiving. When a peer wants to obtain a file, it needs to join such a swarm in order to be able to share with the group.

One big issue with cooperative models like the one just described, is that it relies on peers behaving with 'community spirit'. That is, once they have obtained their complete file, they should make themselves available to continue running the BitTorrent upload, in order to help others obtain the file. The more peers that continue with the uploading, the more efficient the overall system will be. The term **leeching** is used to describe the problem of peers who are interested only in downloading, but who refuse to upload, thus causing a drain on the system. To combat leeching, BitTorrent uses the 'tit-for-tat' reward scheme, as an incentive for peers to continue to be part of the swarm. With tit-for-tat a peer receives a ranking for staying connected to upload to others, which will then give them a better download time the next time they participate in a file-sharing activity. This scheme has proved very successful.

As with Napster, there are social and legal problems associated with BitTorrent, since many users can now obtain pirated versions of films, videos and the like, which worries the movie industry. On the other hand, it looks a promising opening for interactive TV. The notion of being able to watch TV programmes exactly when the consumer wants, becomes feasible if it is possible to quickly download a TV programme.

8.5 Trade-offs

We have now considered a number of different forms of communication, and discussed their characteristics. Here we briefly compare them and ask which type of communication is best under which circumstances. We look at this from the following three angles.

Abstraction versus overhead

Figure 23 shows some forms of communication in terms of a sliding scale of abstraction. In this unit we have seen that increased levels of abstraction, such as provided by RPC and RMI as compared to socket programming, are beneficial because they allow the programmer to concentrate on the bigger picture and are therefore less error-prone, and easier to manage and update.

However, there is also a cost to this increased level of abstraction: *overhead*. You saw that RPC requires an additional layer of software, the RPC service layer. Even though we have not discussed any of this in detail, we can assume that the communication models higher up in the table are also implemented by adding further software layers. Obviously, the addition of one or more extra layer(s) of software is more resource intensive for the system as a whole, and probably leads to slower communication than the direct use of sockets.

For this reason, sockets may remain the better option for systems that have only limited resources, or for systems where a fast response time is of vital importance.

Scalability

As distributed applications grow, they get more complex to manage. When thinking about the communication between the various parts of a system, it is important to consider how easy/difficult it is to accommodate growth in the system as a whole.

For example, when more and more participants are added to a system, such as an auctioning system, it may be difficult to manage the addressing details for all the clients, and to manage sending the right message to the right clients at the right time. The publish/subscribe system handles a lot of the complexity of managing the participants in the system and may be the right choice under such circumstances.

For applications that are increasingly heavy in their bandwidth requirements, such as video or audio data, it may be appropriate to look at communication protocols for peer-to-peer systems, such as the BitTorrent protocol.

Cross-platform support

We have seen that middleware, such as RMI and ORBs, adds a software layer to the system to mask the heterogeneity of the underlying platform. However, you also learnt that RMI is Java specific, and can be used only when all the objects, distributed and local, are written in Java, whereas ORBs can handle objects written in different programming languages.

You have not really seen enough of these different forms of middleware to make an informed judgement at this stage, but you will hopefully learn that all choices come with a cost. If we can assume that all parts of the system are written in one language, say Java, then using a language-homogeneous form of middleware will be simpler and more efficient. Obviously, if that is not the case, middleware that supports language heterogeneity offers greater flexibility.

Exercise 4

A newspaper group has made its news stories available for web users and users of various mobile devices. They now want to run a specialised podcasting service, where groups of readers can be kept informed of the latest news, and receive updates on developments for particular news stories. Explain which form of communication would be best for this.

Discussion

The publish/subscribe paradigm would be particularly well suited for this scenario. The news items would be the published items, and readers can register their interest for such a news story by becoming a subscriber. Each time a development takes place that the newspaper wants to report on, this will be forwarded to all the subscribed users.

9

Summary

In this unit we discussed the client–server architecture for distributed systems, including a number of variations on the classic model: the server as a client, multiple clients working together to provide a service, and proxy servers. We then looked at the peer-to-peer architecture, and compared the architectures in terms of their ability to provide fault tolerance and to improve performance.

There has been a historical development in going from two-tier architectures to n-tier architectures. The latter provide more scalable systems, as well as encouraging reuse, better data integrity, and better security and availability. The Java EE software framework is an example of an n-tier architecture, and specifies the APIs for the components within this framework.

Message passing is the most basic form of communication in a distributed system. Message passing can be either synchronous or asynchronous and, if asynchronous, a buffer is required so that the sending process can deposit its message there. If the data that is passed around is more complex than, for example, a simple integer, it may need to be marshalled into some form suitable for transmission, and unmarshalled upon arrival at the receiving process.

Message passing can be implemented in Java programs using the `java.net` package. A server process will continuously listen for processes that attempt to connect to it, and, once this occurs, a connection in the form of a pair of sockets is set up between the server and client processes. Using the input and output streams of the sockets, the processes communicate. Servers are usually multithreaded in order to be able to communicate with a number of clients concurrently.

Socket programming can be cumbersome when dealing with more complicated scenarios, and higher-level paradigms provide better support to the developer to implement the communication. We discussed remote procedure calling as a form of task-oriented communication, with Remote Method Invocation as its object-oriented variant. At an even higher level of abstraction are the messaging system model and the object request broker, which both operate by using an intermediary for the sending and receiving processes, rather than allowing the processes to communicate with each other directly.

LEARNING OUTCOMES

When you have completed your study of this unit you should be able to do the following:

- ▶ distinguish the main architectures for distributed systems, and discuss strengths and weaknesses of each architectural model;
- ▶ be aware of the importance of performance and fault tolerance issues when comparing distributed system architectures;
- ▶ compare and contrast the two-tier architecture with the n-tier architecture and explain the main functionality of each tier in the Java EE platform;
- ▶ distinguish between synchronous and asynchronous message passing, and explain the need for data marshalling and unmarshalling for communication in distributed systems;
- ▶ use the Java `Socket` API to implement simple client-server systems;
- ▶ compare and contrast a number of different models for communication in distributed systems and explain the core issues involved.

Glossary

action-oriented model A software model in which the actions carried out by processors are central, as opposed to an object-oriented model where objects are central, or a **data-oriented model** where data is central.

addressing Identifying a particular computer on the internet using either a symbolic address or a numeric address.

application programming interface (API) An interface provided by a computer system or program library. Services specified in a computer system/library can be made available to, and called on by, computer programs. The software that provides the functionality described by an API is said to be an implementation of the API.

application server A server dedicated to running a number of applications.

asynchronous message passing A form of communication in which the sending and receiving processes do not have to be at their synchronisation points at the same time. This form of message passing requires a buffer in order to store messages in the case of the receiving process not being ready to receive the message.

BitTorrent protocol A protocol for file sharing among peers (in peer-to-peer systems), where a large data file is divided into smaller parts that are distributed among the peers. The peers cooperate by sharing the parts they have received with other peers, such that eventually all peers have all the parts of the original file.

blocking When invoking methods, or calling procedures, a process is said to be blocked for the duration of the method call.

business logic Refers to the logic that embodies business rules rather than the view/presentation or storage of data. Typically, this would be the calculations and processes appropriate for the business.

caching A technique of temporarily storing frequently accessed data in a special area to reduce the time required to gain access to the data.

client-server A software architecture style in which asymmetric roles are assigned to two collaborating processes. One process, the server, plays the role of a service provider which waits passively for the arrival of a request. The other, the client, issues specific requests to the server and awaits its response.

component model Definition of standards for the implementation, documentation and deployment of components. Examples of such models are the Java EE framework, EJB and CORBA.

data partitioning Dividing the data resources of a system and storing them on several servers which are distributed. Each server is responsible for maintaining its own partition.

data replication Duplication of data and the storing of data on several servers such that multiple backup copies exist.

data-oriented model A software model in which the focus is on data, and where actions are triggered as a result of data being exchanged. See also **action-oriented model**.

deployment The placing or installing of software on a target system so that it is ready to run. This process typically involves some configuration steps to tailor the software to the specific organisation, users, or application domain – for example, to specify location-specific information, such as a list of local users permitted to access the software, and the name of the local database.

distributed object paradigm An extension of the object-oriented paradigm to a distributed setting, in which local objects and remote objects are treated in the same way and have the same capabilities.

distribution transparent The requirement that users can remain unaware of some of the complexities of operating in a distributed setting.

event/notification model A model in which an object can generate certain events, and when such an event occurs, it publishes a notification. The notification can be picked up by the objects that subscribed to this event.

fault tolerance The ability of a system to respond gracefully to an unexpected hardware or software failure. By ‘gracefully’ we mean that if a system’s operating quality decreases at all due to a failure, the decrease is proportional to the severity of the failure. This is in contrast to a naively designed system in which even a small failure can cause total breakdown.

heterogeneity Programs running on different computers with different operating systems, data representations and programming languages.

heterogeneous middleware Middleware in which no assumptions are made about a shared language or whether all parties are running on the same platform. CORBA is a heterogeneous form of middleware.

homogeneous middleware Middleware in which all parties are assumed to be implemented in the same language and running on the same platform. For example, **Remote Method Invocation** is a homogeneous form of middleware, in that it assumes that the whole system runs within the Java Virtual Machine environment and that one single programming language (Java) is used throughout.

interface (component/system) Used to refer to the commands or operations (methods), including their syntax and semantics, that a software system or component provides to the clients of the system. Through the interface, the software entity presents methods for external communication, and the interface allows the entity to be internally modified without affecting the way outside entities interact with it.

Internet Protocol (IP) address The numeric address of a device on the internet.

layer A layer of software in which implementation details are hidden and where the functionality of the layer can be invoked through a well-defined interface.

leeching A term used to describe the behaviour of those peers in a peer-to-peer system that cause a drain on the overall system – they wish to benefit only from the system and do not participate in helping other peers.

legacy system A software system that is still required because it meets some useful purpose for an organisation. It is not considered to be replaceable and so is maintained, usually on a restricted budget.

load balancing Balancing the load (the amount of work) in order to avoid having one component doing all the work, while other components remain idle.

local object An object whose methods can be invoked only by a local process.

local process A process that runs on the machine on which the objects it is dealing with reside.

marshalling The process of taking a collection of data items and assembling them into a form (such as a byte stream) suitable for transmission in a message.

messaging system paradigm A form of middleware in which messages are not sent directly from client to server, but go via the messaging system, which acts as the intermediary. The intermediary then looks after the message from that point onwards. Also known as message-oriented middleware (MOM) or the message queuing system. An example is the Java Message Service (JMS).

middleware In a distributed system, the software that interfaces between a client and a specific operating system. It provides a common interface for clients and allows clients to use the same system calls, no matter what operating system is in use.

multicast A means of communication whereby the sender sends a message once to a group of receivers who have previously registered to receive such messages.

object request broker (ORB) Middleware that sits between the client and the objects the client wishes to invoke. The ORB is responsible for locating each object and communicating the client's request to the object. The object carries out the request and replies.

Open Systems Interconnection (OSI) model A layered, abstract description for the design of communication and network protocols.

orphan A remote procedure call whose client (i.e. the node making the call) crashes after the request has been sent.

paradigm A model, example or pattern, especially an outstandingly clear or typical example or archetype.

peer-to-peer (P2P) A distributed system architecture where each computer has similar capabilities and responsibilities – each can act as a server as well as a client.

performance A measurement of how well a system deals with its workload, usually expressed in terms of its responsiveness to users and the throughput of work.

port A numbered conduit into a host on the internet, which allows a client to request a particular kind of service and allows a server to listen for particular kinds of request on a specific port. For example, port 80 is normally used to access a web server.

procedure call When a procedure is called, the code associated with the procedure is executed, and once this is finished and any necessary data has been returned, the execution of the main program continues with the next statement. This is similar to a method invocation.

proxy server A server that acts as an intermediary between clients and other servers.

publish/subscribe messaging model A communication model in which an object that generates events publishes the type of event it makes available for observation by other objects; and objects that want to receive notifications subscribe to events that are of interest to them.

Remote Method Invocation (RMI) A form of middleware, the Java version of remote procedure calling. With RMI, methods can be invoked on objects whether they are local or are remote from the invoking client.

remote procedure call (RPC) A call to a procedure located on a machine that is different from the machine from which the call is sent.

remote process A process running on a different machine from (but connected to) the one on which the object it is dealing with resides.

responsiveness The speed at which a response is generated.

scalability The ability of a system to scale (shrink or grow) in order to support larger or smaller volumes of data and more or fewer users.

ServerSocket A type of object used by a server program in a client–server application. The object has the role of waiting for a client and returning a connection socket once the connection with this client is established.

single point of failure The part of a system that can cause the whole system to go down when a failure occurs in this particular part.

socket An abstraction for each of the two sides of the connection between the client and server. It is identified by a combination of port and host name, and provides the facilities (e.g. input and output streams) to enable data to be transferred between client and server. In Java these are implemented with the `Socket` class.

software architecture A software architecture is concerned with the structures of the system. These structures comprise software elements, the main properties of those elements and the relationships between them.

software component An independent software unit that can be composed with other components to create a software system. Each component has a well-defined interface through which the services of the component can be invoked, while shielding its implementation details.

swarm A group of connected peers working together in a peer-to-peer system.

synchronous message passing A form of communication in which the sender waits (i.e. it is blocked) until it receives a response from the receiver. See also **asynchronous message passing**.

thick client Complex client software that carries out significant processing on the client machine in a client–server system.

thin client In a client–server system, a simple client with very limited functionality, mainly limited to display and communication – for example, a web browser.

throughput The rate at which computational work is done.

tier The common functionality of computer processes or components at a very high conceptual level.

timeout The expiry of a preset time interval causing a thread to stop running because it has used its allocated execution time.

two-tier model A software architecture consisting of a database tier and a client applications tier.

unicast The sending of information to a single destination.

unmarshalling The process of extracting a data structure from a series of bytes.

References

- Bishop, J. (1998) *Java Gently – Programming Principles Explained* (2nd edn), Addison-Wesley.
- Capra, P. (2004) *Peer-to-Peer – Napster* [online],
http://wiki.media-culture.org.au/index.php/Filesharing_and_P2P-Napster
(Accessed 9 October 2007).
- Lewis, D. (2007) *James Bond Meets the 7 Layer OSI Model* [online],
<http://www.lewistech.com/rlewis/Resources/JamesBondOSI2.aspx> (Accessed 9 October 2007).
- Sun Microsystems (2007) *Java EE 5 Tutorial* [online],
<http://java.sun.com/javaee/5/docs/tutorial/doc/bnaay.html> (Accessed 9 October 2007).

Acknowledgement

Grateful acknowledgement is made to the following source.

James Bond and the OSI Model (Subsection 3.2): Courtesy of Richard Lewis,
www.lewistech.com.

Index

A

- abstraction 47–48, 55, 59
- addressing 31
 - numeric 31
 - symbolic 31
- application programming interface (API) 18, 26
- application server 20
- availability (of a server) 21

B

- BitTorrent protocol 57
- blocking 29–30
- Bond, James 17
- buffer 29
- business logic 20

C

- caching 11
- chat room 39
- client
 - thick 15, 22
 - thin 21–22
- communication 28, 55
 - abstraction versus overhead 59
 - basic message passing 28
 - group 57
 - messaging system paradigm 56
 - paradigm 47
- component model 26
- configuration 26

D

- data
 - integrity 21
 - partitioning 9
 - replication 9
- deadlock 30
- deployment 26
- distributed
 - objects 51, 55
 - system 6
- distributed object paradigm 51
- distribution transparency 47

E

- enterprise 23–24
- Enterprise JavaBeans (EJB) 26–27, 52
- Ethernet 17

F

- failure 8, 10, 51
 - RPC 51
- fault tolerance 7, 10
- flattening data 32

G

- graphical user interface (GUI) 19

H

- heterogeneity 32
- heterogeneous objects 56

I

- InetAddress 36
 - getLocalHost method 36
- input stream 35
- interface (component/system) 18
- Internet Protocol (IP) address 31, 38
- inter-process communication (IPC) facility 29, 32

J

- Java Enterprise Edition (Java EE) 23, 25–26, 52
 - Java EE server 26
- Java Message Service (JMS) 57
- java.net package 36
- JavaServer Pages (JSP) 26

L

- layer 16
- legacy system 24
- load balancing 8, 13
- local
 - machine 48
 - object 51
 - process 51
- lock (class owned) 42

M

- marshalling 33, 50
- message passing 32, 35
 - asynchronous 28
 - external representation of data 32
 - synchronous (blocking) 29
- messaging model
 - event/notification 56
 - publish/subscribe 56
- messaging system paradigm 56
- method invocation 29, 47
- middleware 49
 - heterogeneous 56
 - homogeneous 52
 - messaging system (MOM) 56
 - messaging system paradigm 56
 - object request broker 56
 - RMI 52
 - RPC 49
- multicast 57
- multiple threads 29, 43
- multithreaded server 45

N

- Napster 12, 57

O

- object request broker (ORB) 55, 59
- Open Systems Interconnection (OSI) model 16
 - RPC service layer 49
- orphan (in RPC) 51
- output stream 35

P

- paradigm 47
- peer-to-peer (P2P)
 - BitTorrent 57
 - file sharing 57
 - leeching 58
 - swarm 58
 - tit-for-tat reward scheme 58
- performance 7, 9
- podcasting 60
- port 31, 38
- procedure call 47
- proxy server 10

R

- receive* operation 29
 - non-blocking 29
- redundancy 21
- remote
 - machine 48
 - object 55
 - process 51
- Remote Method Invocation (RMI) 52
- remote procedure call (RPC) 47–48, 59
 - implementation 50
 - service layer 49
 - timer 50
- responsiveness 7
- reuse 21

S

- scalability 15, 21, 59
- security 21
- ServerSocket 36–37
 - accept method 37
- servlet 27
- single point of failure 8, 13
- socket 35–36, 38, 48
 - input stream 35
 - output stream 35
- Socket 36, 39
- software architecture 7
 - client-server 7
 - layered 16
 - multiple servers 7
 - n-tier 19
 - peer-to-peer (P2P) 12
 - proxy servers 11
 - two-tier 14
- software component 18
- software model
 - action-oriented 49
 - data-oriented 49

T

- TCP/IP protocol 17
- threading 29
- throughput 7
- tier 15, 21, 25
 - business 25
 - client 14, 26

database 14, 20, 25
enterprise information system (EIS) 25
middle 20
presentation 21
web 25–26
timeout 30, 50

U
unicast 57
unmarshalling 33, 50